

Projet *Réservation de salles* : Conception v1.0

1. Table des matières

Projet <i>Réservation de salles</i> : Conception v1.0	1
1. Table des matières	1
2. Objectif du document	2
3. Architecture	2
3.1. Choix des technologies	2
3.2. Contraintes techniques	3
4. Technologies utilisées	3
4.1. Serveur web	3
4.2. Stockage des données	3
4.3. Couche de persistance	3
4.4. Couche métier	3
4.5. Couche service et API REST	3
4.6. Couche présentation	4
4.7. Authentification	4
4.8. Environnement de développement	4
4.9. Tests	4
4.10. Packages et dépendances	4
4.11. Déploiement	5
5. Préliminaire à la conception	7
6. Cas d'utilisation	8
6.1. Rappel : modèle trouvé en analyse	8
6.2. Groupe 1 : Gestion des comptes	8
6.3. Groupe 2 : Gestion des salles	11
6.4. Groupe 3 : Gestion des réservations	16
6.5. Groupe 4 : Authentification	22
6.6. Groupe 5 : Gestion de la liste d'attente	23
7. Regroupement des classes	26
7.1. Groupe domaine	26
7.2. Groupe repositories	26
7.3. Groupe services	27
7.4. Groupe contrôleurs REST	27
8. Choix, questions ouvertes et remarques	27

2. Objectif du document

Ce document aborde l'architecture, la conception et les choix techniques pour l'implémentation du projet "Réservation de salles". Les diagrammes suivent le langage de modélisation UML et la méthodologie Arrington.

- On commencera par énumérer les diverses contraintes techniques qui pèsent sur notre projet ;
- on décrira ensuite les technologies choisies ;
- puis l'architecture ;
- et enfin, nous décrirons le design de notre système en revenant sur les *use cases*.

3. Architecture

3.1. Choix des technologies

Le système est découpé en deux parties distinctes : un **backend** Spring Boot exposant une API REST, et un **frontend** Vue.js qui consomme cette API. Ce choix découple clairement la logique métier de la présentation, ce qui facilite la maintenance et l'évolution indépendante de chaque partie.

Les points à considérer sont en particulier :

- la complexité de l'interface utilisateur ;
- les contraintes de déploiement ;
- le nombre et le type d'utilisateurs ;
- la sécurité ;
- les performances et le passage à l'échelle.

3.1.1. Administration du système

- opérations essentiellement CRUD (gestion des comptes, des salles, des réservations, etc) ;
- l'administrateur est un utilisateur interne ;
- sécurité importante : accès réservé aux administrateurs.

Une interface web classique est suffisante. La même application Vue.js pourra proposer des vues spécifiques aux administrateurs, protégées par le système d'authentification.

3.1.2. Côté utilisateurs

- les utilisateurs consultent et filtrent la liste des salles disponibles ;
- ils effectuent, modifient et annulent des réservations ;
- ils reçoivent des notifications par mail ;
- une interface web responsive est souhaitée.

L'application Vue.js fournit une interface fluide sans rechargement de page complet. Les interactions avec l'API REST permettent de filtrer les salles dynamiquement et de gérer les réservations en temps réel.

3.1.3. Côté responsables

- les responsables gèrent les salles, les équipements et les plages de disponibilité ;
- ils valident ou rejettent les demandes de réservation ;

- l'interface est plus riche que celle des utilisateurs simples, mais reste dans les capacités d'une SPA Vue.js.

3.2. Contraintes techniques

- le système doit être accessible de l'extérieur via HTTPS ;
- le backend expose une API REST ; le frontend Vue.js est une application distincte qui consomme cette API ;
- le système doit être fiable pour l'envoi de mails (confirmation de réservation, annulation, etc) ;
- l'application doit être raisonnablement sécurisée (authentification JWT, HTTPS, validation des données côté serveur).

Deux architectures de déploiement sont envisagées, décrites en détail en section 4.11 :

- **Architecture simple** (cible initiale) : un seul conteneur backend, un seul conteneur frontend, une seule base de données PostgreSQL. Suffisante pour un établissement d'enseignement à charge modérée. C'est cette architecture qui sera mise en place en premier.
- **Architecture avec load balancer** (évolution possible) : si la charge augmente, on peut passer à une architecture horizontalement scalable avec un load balancer devant plusieurs instances du backend. L'utilisation de **tokens JWT sans état** (*stateless*) facilite cette évolution : aucune session côté serveur n'est à partager entre les instances.

4. Technologies utilisées

4.1. Serveur web

Pour des raisons de facilité de maintenance, on choisit d'utiliser une application **Spring Boot** hébergée dans un conteneur Docker. Le serveur utilisé sera un serveur **Tomcat embarqué**. Le backend expose uniquement des endpoints REST (pas de rendu de templates côté serveur).

4.2. Stockage des données

Les données seront stockées dans une base **PostgreSQL**, aussi bien en développement qu'en production. Ce choix garantit la cohérence entre les environnements et évite les comportements inattendus liés aux différences entre bases de données.

4.3. Couche de persistance

La couche de persistance sera **JPA / Hibernate**, que l'équipe maîtrise bien. Les entités seront annotées JPA, et les repositories seront des interfaces Spring Data JPA.

4.4. Couche métier

La couche métier sera composée d'entités JPA et d'implémentations de services Spring (**@Service**). Les règles métier importantes (vérification de disponibilité, gestion des conflits de réservation, transitions d'état) seront encapsulées dans les entités ou dans des classes de service dédiées.

4.5. Couche service et API REST

La couche service est isolée de la couche présentation via des **DTOs**. Les **contrôleurs REST** (**@RestController**) exposent les endpoints JSON, et délèguent le traitement aux services. Cette

séparation permet de tester les services indépendamment des contrôleurs.

4.6. Couche présentation

La couche présentation est entièrement gérée par **Vue.js**, une application SPA (Single Page Application). Elle communique avec le backend via des appels HTTP (Axios ou Fetch). Le build Vue.js produit des fichiers statiques (HTML, CSS, JS) qui sont servis par **Nginx** directement, indépendamment du backend Spring Boot. Ce choix est cohérent avec l'architecture horizontalement scalable retenue : les instances Spring Boot ne gèrent que l'API REST, et Nginx reste le seul point de distribution du frontend.

4.7. Authentification

L'authentification sera gérée par **Spring Security** avec des **tokens JWT**. Le frontend envoie ses identifiants, reçoit un token, et l'inclut dans l'en-tête **Authorization** de chaque requête. Ce mécanisme est adapté à une architecture découplée frontend/backend.

On pourra envisager dans un second temps l'utilisation de **Keycloak** pour une gestion plus avancée des identités.

4.8. Environnement de développement

Les projets backend seront manipulés via **Gradle**. Le projet frontend utilisera **npm** avec **Vite** comme outil de build. Les développeurs utilisent l'IDE de leur choix.

4.9. Tests

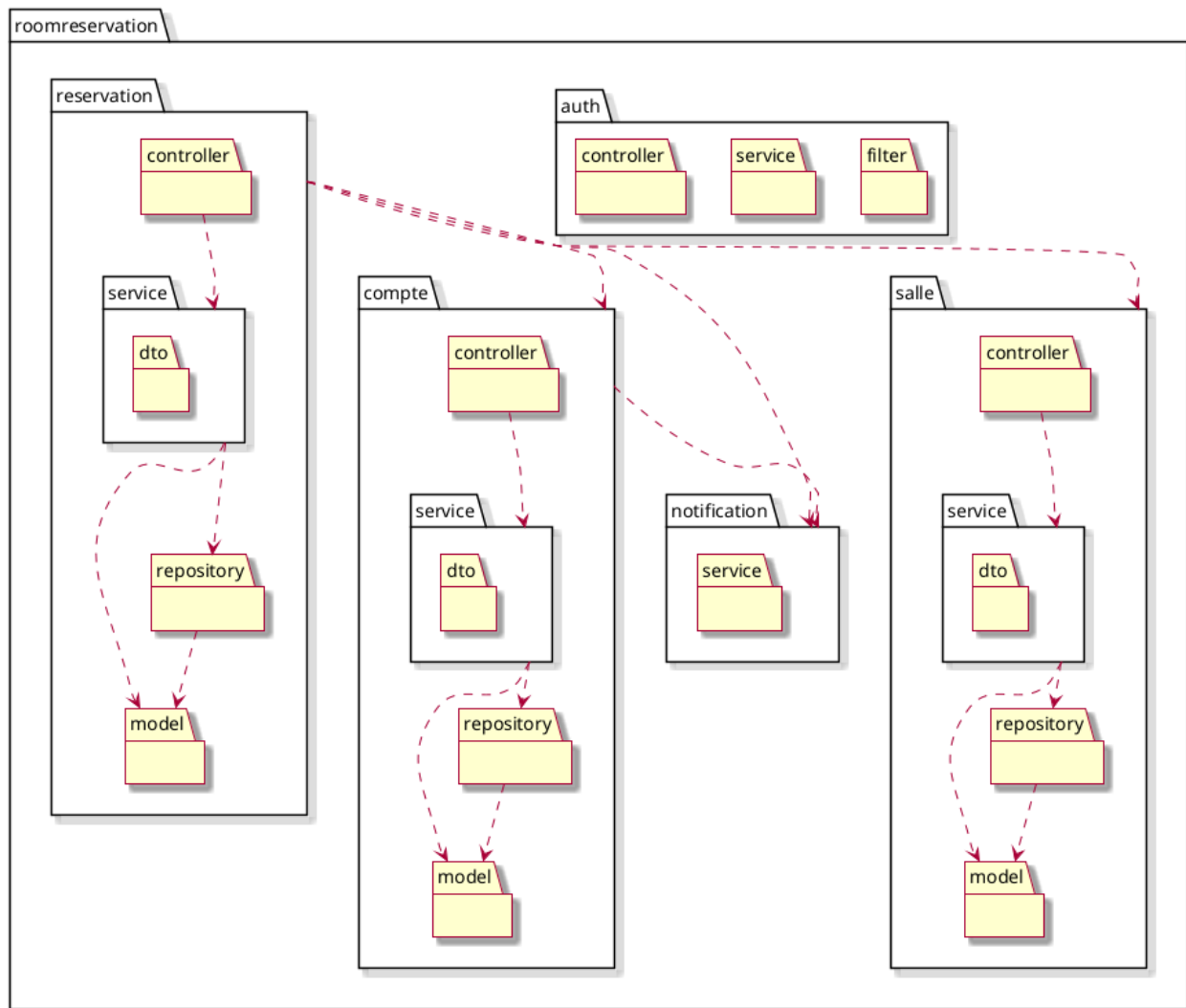
- Tests unitaires backend : **JUnit** ;
- Tests d'intégration : **Spring Boot Test** avec base PostgreSQL (via **Testcontainers**) ;
- Tests frontend : **Vitest** (ou Jest) pour les composants Vue.

4.10. Packages et dépendances

L'approche **Acteur/Action** permet de proposer les composants backend suivants :

- **compte** : gestion des demandes de création de compte, validation, refus, et gestion des informations utilisateur ;
- **salle** : gestion des salles, des équipements et des plages de disponibilité ;
- **reservation** : cycle de vie des réservations (création, validation, rejet, annulation, liste d'attente) ;
- **notification** : envoi de mails (confirmation de réservation, annulation, validation de compte) ;
- **auth** : authentification et gestion des tokens JWT.

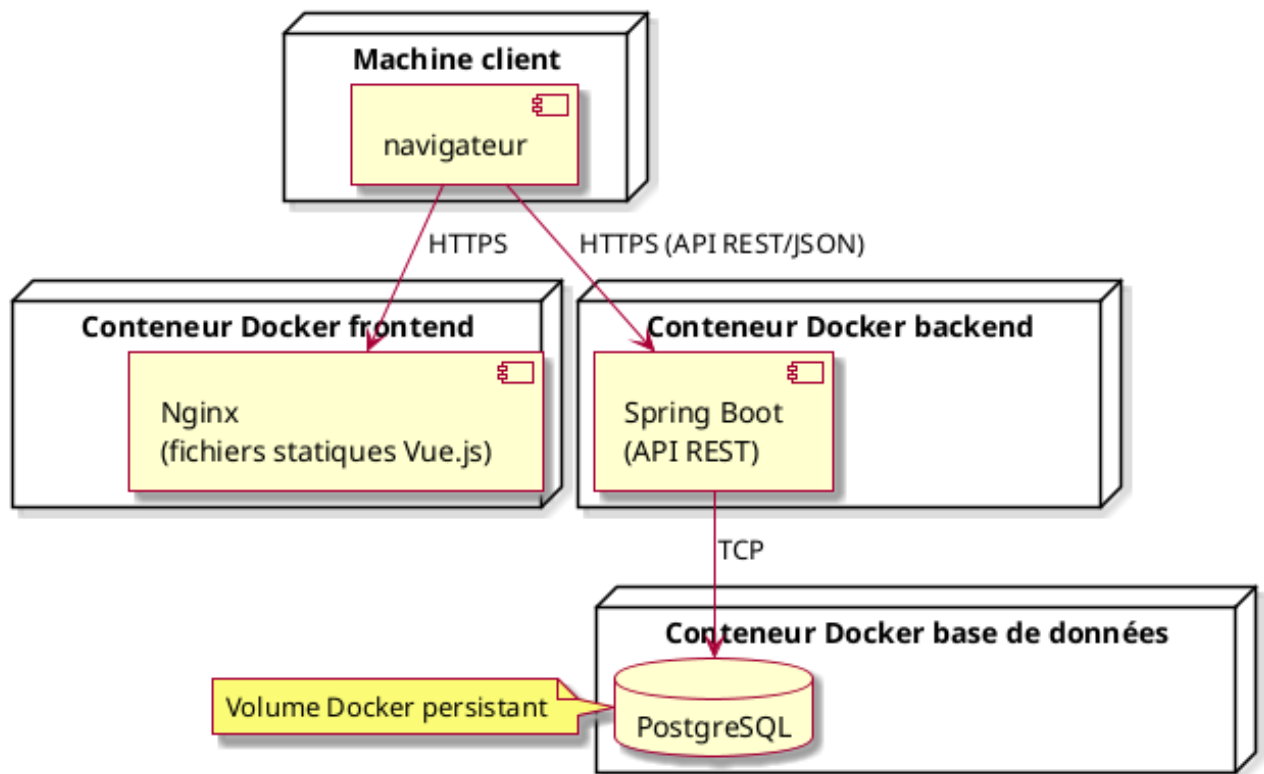
Chaque composant suit la même structure interne :



À chaque composant correspond un (ou plusieurs) `@RestController` qui sert de **façade** et communique avec la couche de présentation à travers des **DTOs**. Les entités JPA ne sont jamais exposées directement dans l'API.

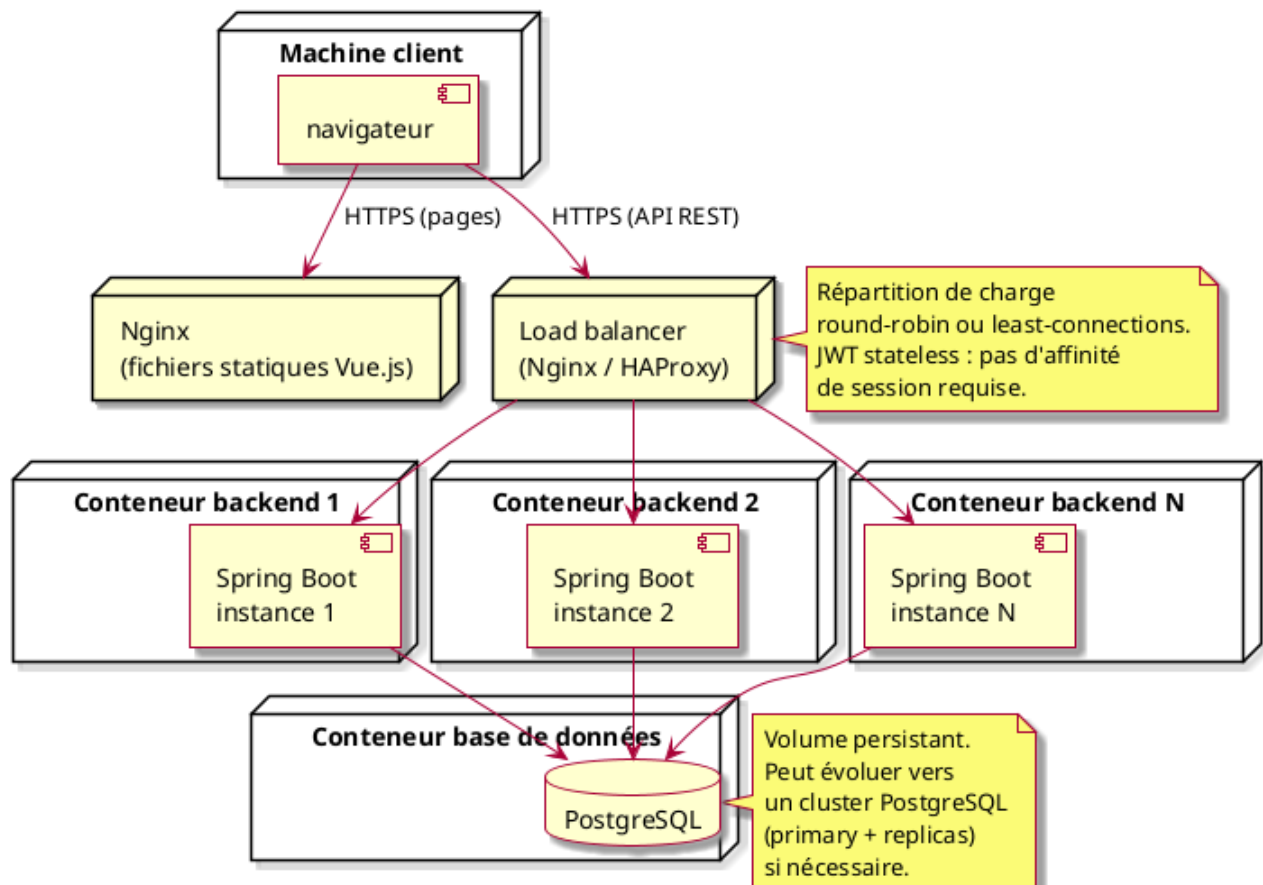
4.11. Déploiement

4.11.1. Architecture simple (cible initiale) Un seul conteneur par composant : frontend, backend, base de données. Orchestration via **Docker Compose**.



- le frontend Vue.js est servi par **Nginx** comme fichiers statiques ;
- le backend Spring Boot expose l'API REST sur un port dédié ;
- la base de données PostgreSQL est dans son propre conteneur avec un volume persistant ;
- les trois conteneurs sont orchestrés via **Docker Compose**.

4.11.2. Architecture avec load balancer (évolution horizontale) Si la charge augmente, on peut multiplier les instances backend derrière un **load balancer**. Le frontend reste servi par un **Nginx** unique, les fichiers statiques Vue.js n'ont pas besoin d'être scalés. L'authentification **JWT** (*stateless*) rend cette évolution naturelle : aucune session serveur à synchroniser entre les instances.



- le frontend Vue.js est servi par un **Nginx** unique, les fichiers statiques étant identiques pour tous les utilisateurs, il n'y a aucun intérêt à les dupliquer ;
- le **load balancer** distribue les appels API REST entre les instances Spring Boot ;
- les tokens **JWT étant sans état**, n'importe quelle instance peut traiter n'importe quelle requête sans partage de session ;
- la base de données PostgreSQL reste centralisée ; elle peut évoluer vers un cluster (primary + replicas en lecture) si elle devenait un goulot d'étranglement ;
- cette architecture peut être gérée avec **Docker Swarm** ou **Kubernetes** selon les besoins.

5. Préliminaire à la conception

La conception est un processus itératif. Avant de traiter les *use cases* un par un, il est utile de faire un premier passage sur les classes métier identifiées en analyse, afin d'anticiper les difficultés.

Les entités principales identifiées lors de l'analyse sont :

- **Compte** : représente un compte utilisateur dans le système ;
- **DemandeCreationCompte** : représente une demande de création de compte en attente de validation ;
- **Salle** : représente une salle pouvant être réservée ;
- **Equipement** : représente un équipement disponible dans une salle ;
- **PlageDisponibilite** : représente une plage horaire durant laquelle une salle est disponible ;
- **Reservation** : représente une réservation effective ;
- **DemandeReservation** : représente une demande de réservation en attente de traitement.

À cette liste, la conception ajoute une entité :

- **ListeAttente** : représente une position dans la liste d'attente pour une salle (gestion des désistements sur les salles à réservation directe).

Un point important à noter : la gestion de la **disponibilité des salles** et la **détection des conflits de réservation** constituent la complexité principale de ce projet. Il faudra traiter ces *use cases* en priorité pour valider le modèle.

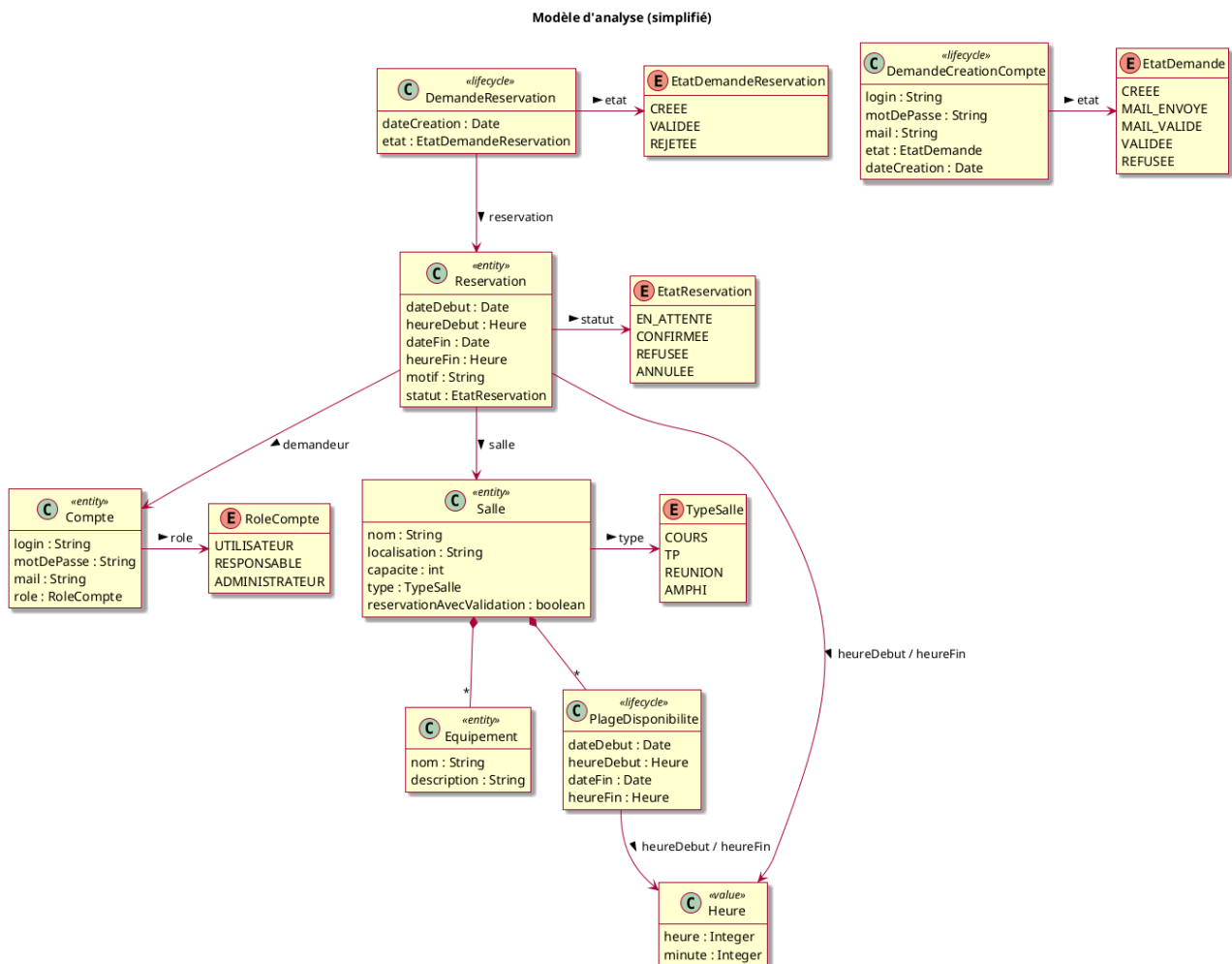
Un second point concerne le **double mode de réservation** : certaines salles sont réservées directement (premier arrivé, premier servi avec liste d'attente), d'autres nécessitent la validation d'un responsable. Ce choix influe sur le cycle de vie de la **Reservation** et son état.

6. Cas d'utilisation

On choisit de développer un sous-ensemble cohérent des fonctionnalités, en commençant par les cas prioritaires identifiés en analyse.

6.1. Rappel : modèle trouvé en analyse

Le modèle issu de l'analyse est un point de départ qui sera affiné pendant la conception.



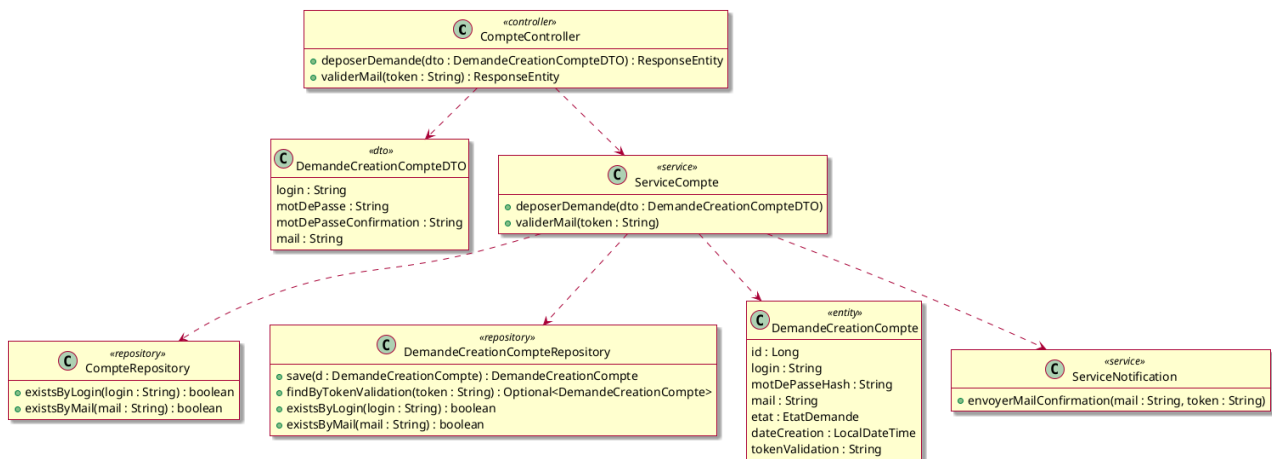
6.2. Groupe 1 : Gestion des comptes

Les quatre cas de gestion des comptes s'appuient sur **CompteController** et **ServiceCompte**. La persistance est assurée par **CompteRepository** et **DemandeCreationCompteRepository**.

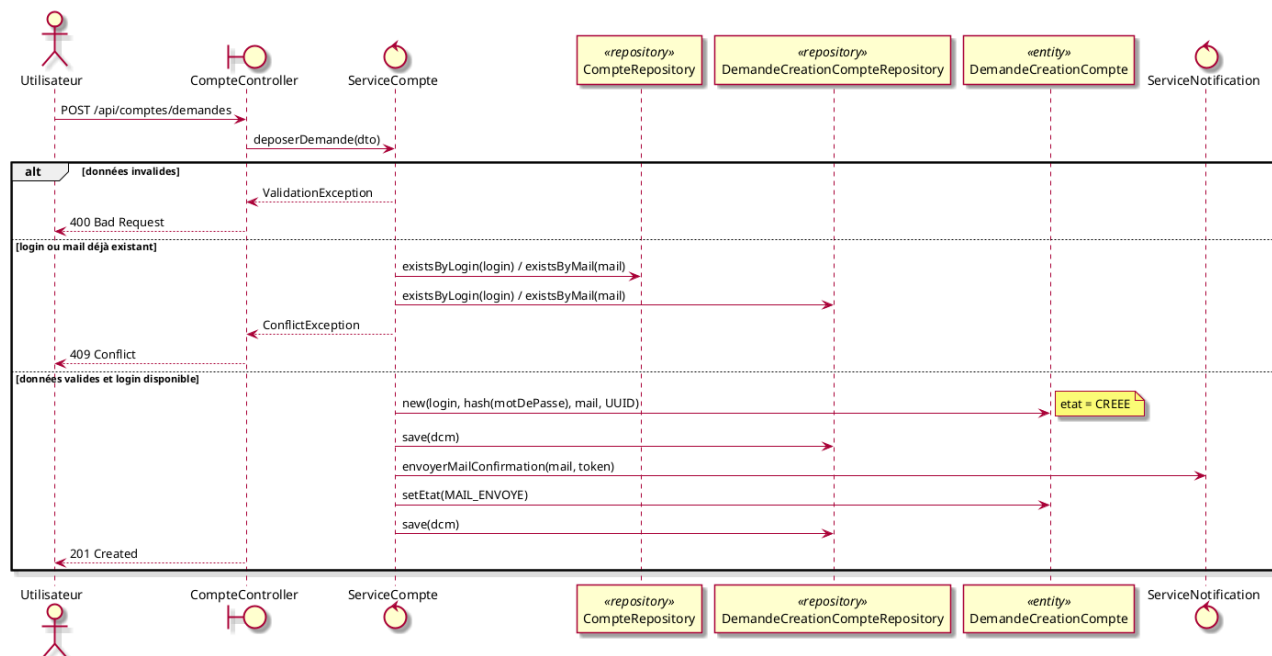
6.2.1. Déposer une demande de création de compte Endpoints : - POST /api/comptes/demandes : soumettre la demande - GET /api/comptes/demandes/valider?token={token} : valider le mail via le lien reçu

Un champ `tokenValidation` est ajouté à `DemandeCreationCompte` pour sécuriser le lien de validation envoyé par mail. Le mot de passe est haché (BCrypt) avant la persistance. Lorsque l'utilisateur clique sur le lien reçu, `validerMail` retrouve la demande via son token et la fait passer à l'état `MAIL_VALIDE` ; elle devient alors visible des administrateurs.

Classes de conception

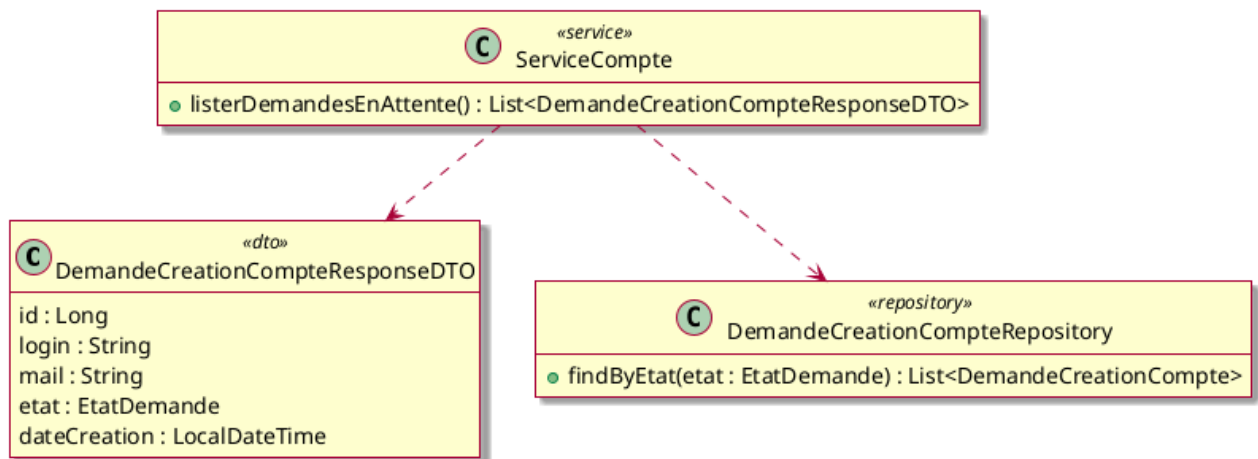


Séquence

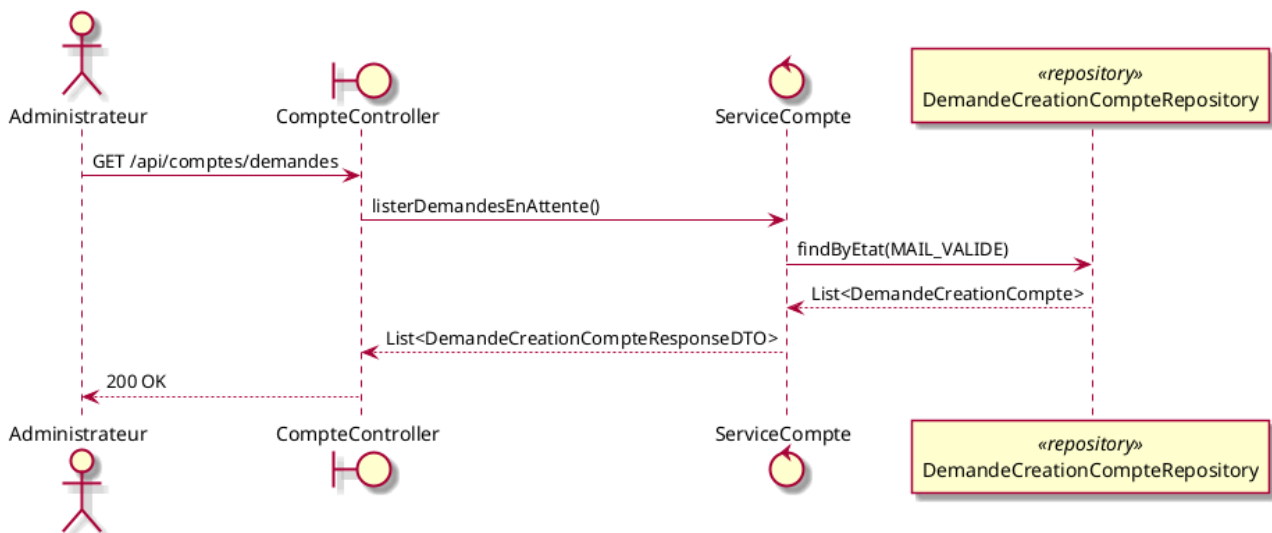


6.2.2. Consulter les demandes de création de compte en attente Endpoint : GET /api/comptes/demandes (accès restreint aux administrateurs ; retourne les demandes en état `MAIL_VALIDE`)

Classes de conception Aucune nouvelle classe. On ajoute un DTO de réponse et une méthode au repository.



Séquence

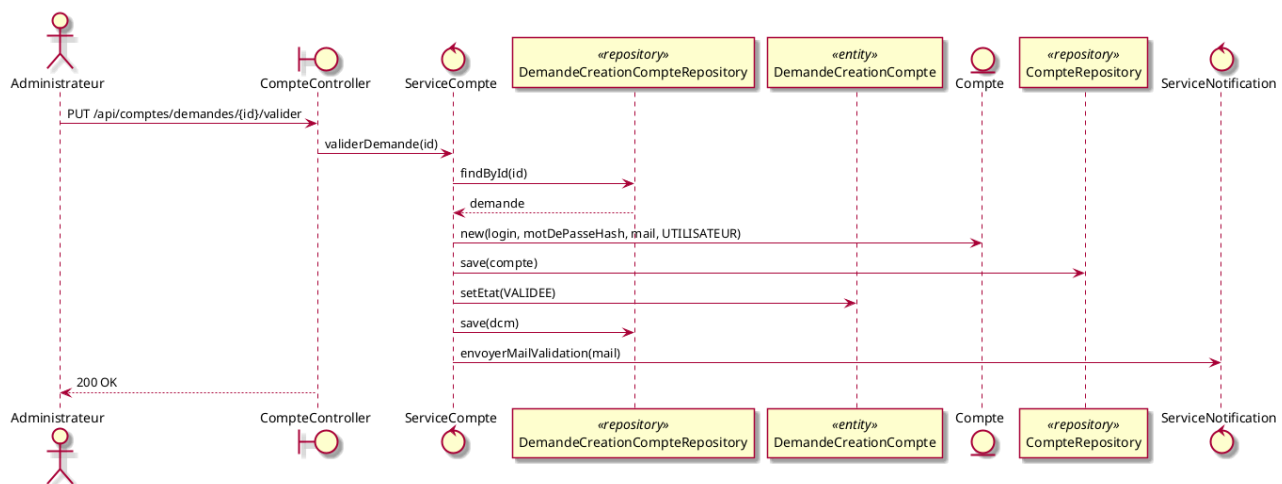


6.2.3. Valider une demande de création de compte Endpoint : PUT /api/comptes/demandes/{id} (accès restreint aux administrateurs)

La validation crée un **Compte** à partir des données de la **DemandeCreationCompte**, puis passe la demande à l'état **VALIDEE**.

Classes de conception Aucune nouvelle classe. On utilise **CompteController**, **ServiceCompte**, **DemandeCreationCompteRepository**, **CompteRepository**, **Compte** et **ServiceNotification**.

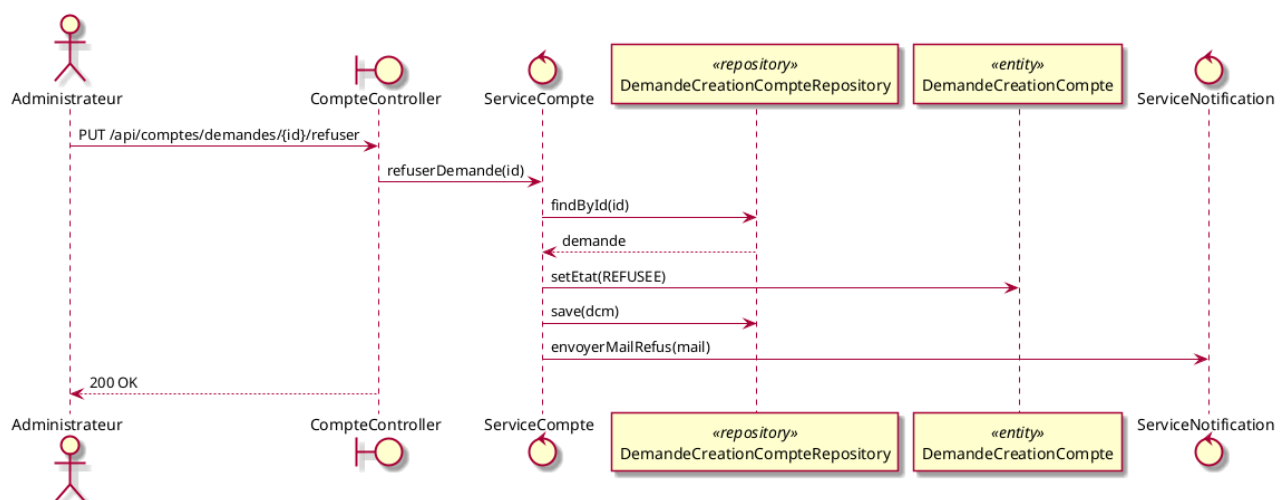
Séquence



6.2.4. Refuser une demande de création de compte Endpoint : PUT /api/comptes/demandes/{id}/refuser (accès restreint aux administrateurs)

Classes de conception Aucune nouvelle classe.

Séquence

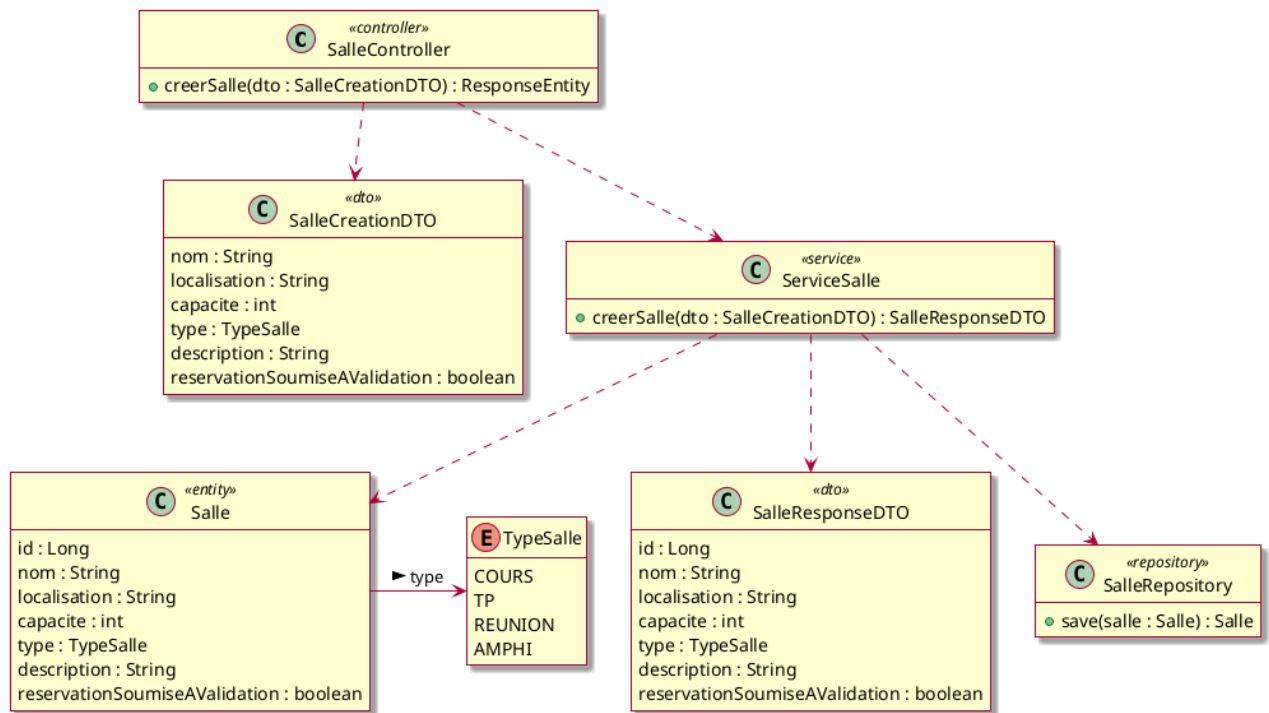


6.3. Groupe 2 : Gestion des salles

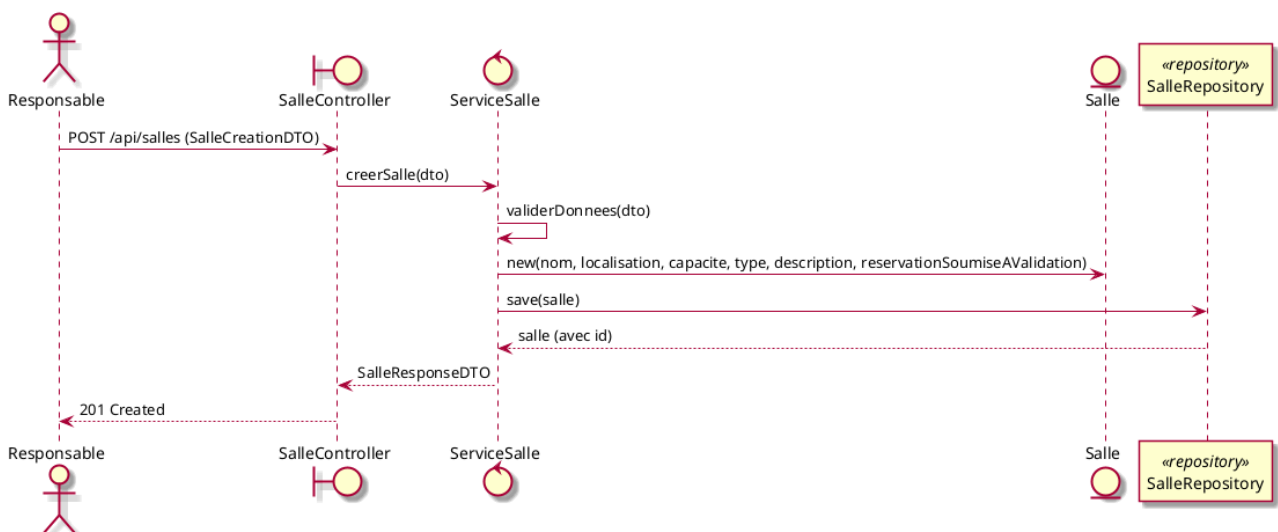
Les quatre cas s'appuient sur **SalleController** et **ServiceSalle**. La persistance est assurée par **SalleRepository**, **EquipementRepository** et **PlageDisponibiliteRepository**.

6.3.1. Créer une salle Endpoint : POST /api/salles (accès réservé aux responsables)

Classes de conception



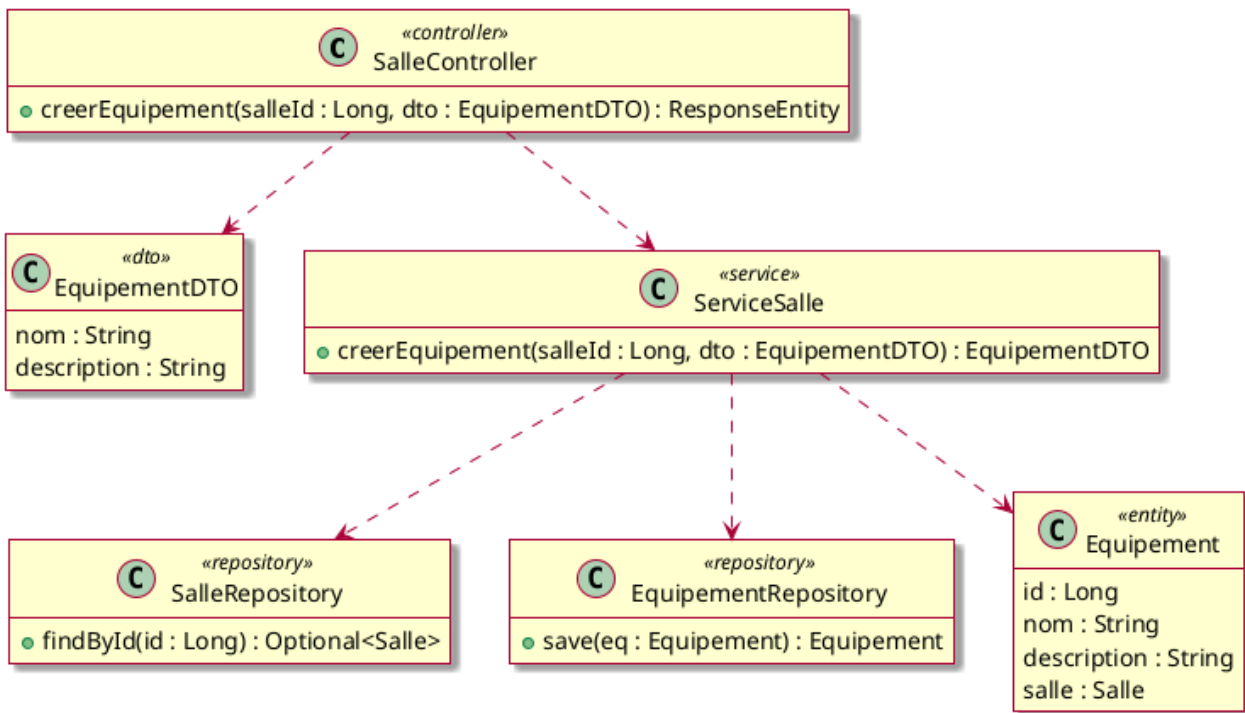
Séquence



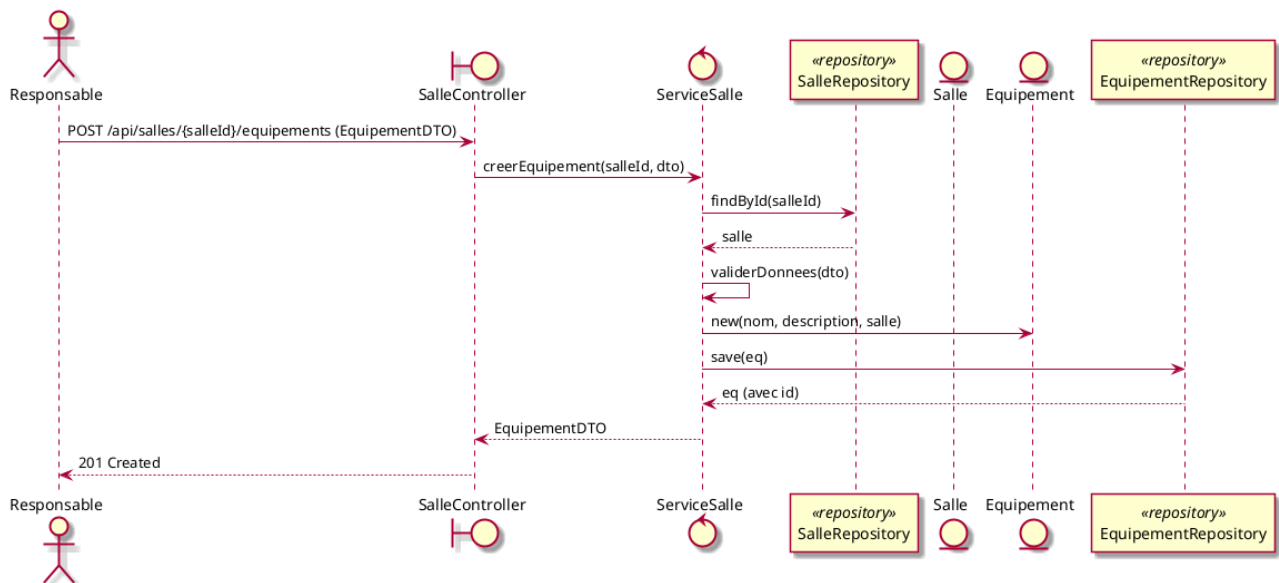
6.3.2. Créer un équipement Endpoint : POST /api/salles/{salleId}/equipements (accès réservé aux responsables)

L'équipement est rattaché à une salle existante via `salleId`.

Classes de conception



Séquence

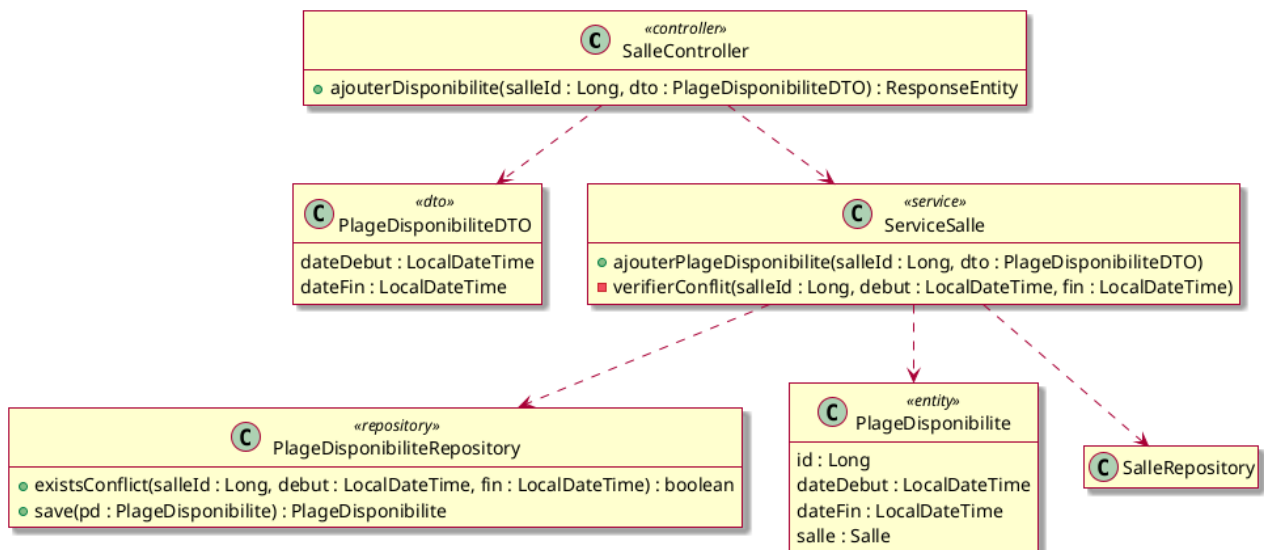


6.3.3. Ajouter une plage de disponibilité d'une salle Endpoint : POST /api/salles/{salleId}/disponibilites (accès réservé aux responsables)

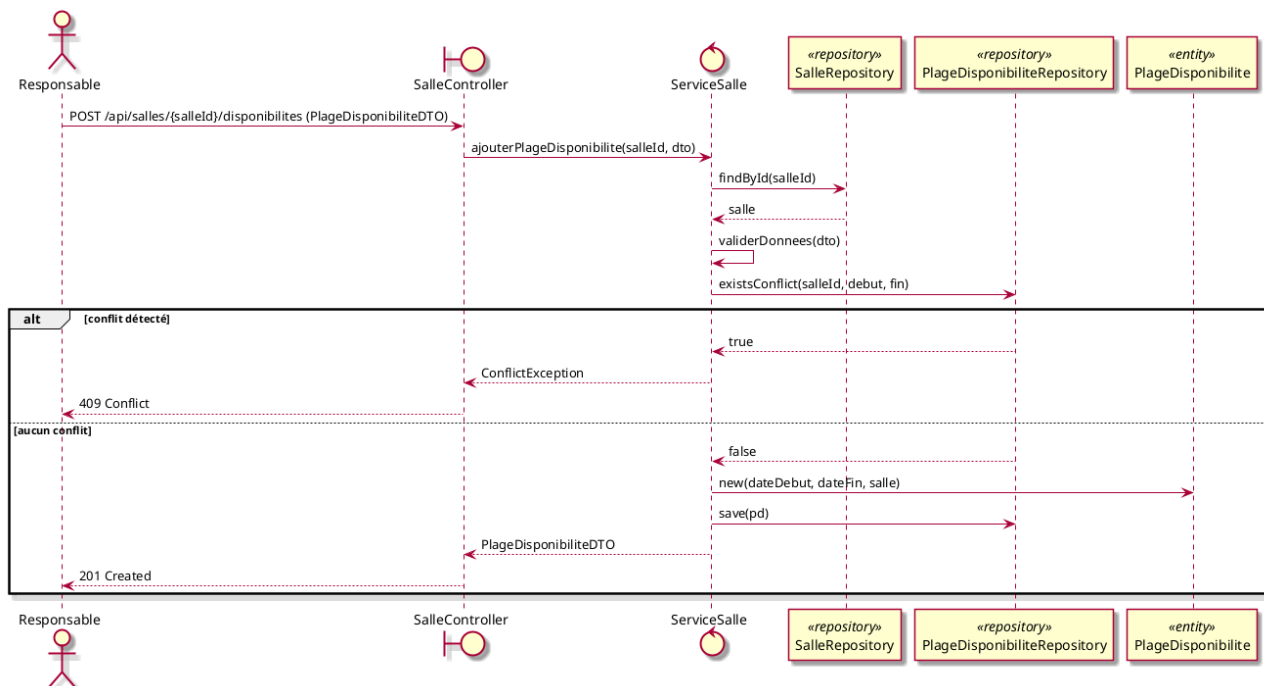
La vérification des conflits est une opération clé : on s'assure que la nouvelle plage ne chevauche aucune plage existante de la même salle. La requête de détection utilise le prédicat :

```
dateDebut < :fin AND dateFin > :debut AND salle.id = :salleId
```

Classes de conception

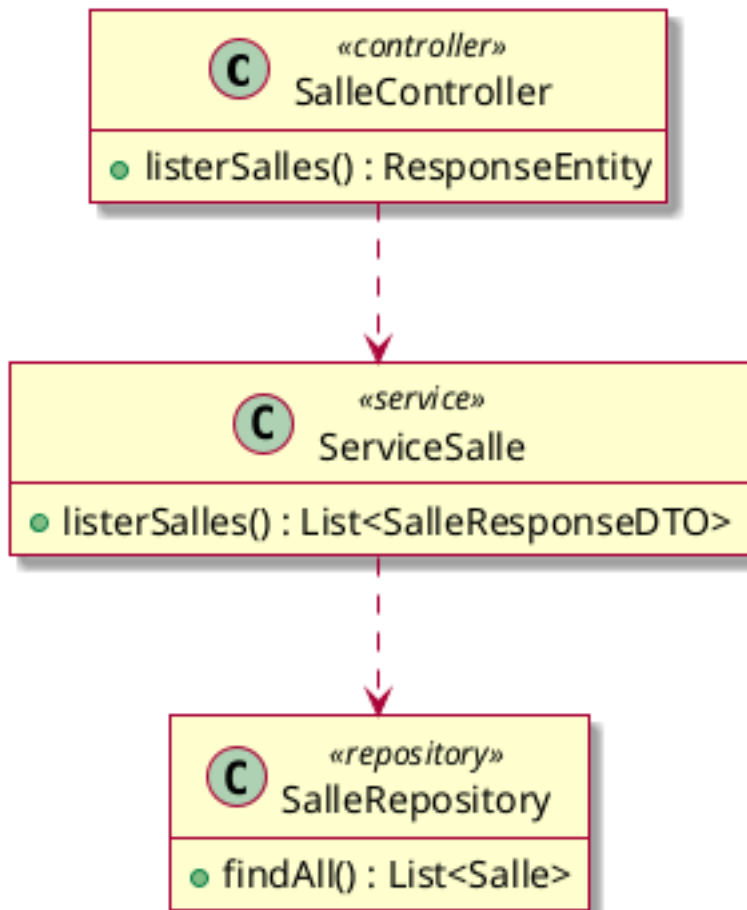


Séquence

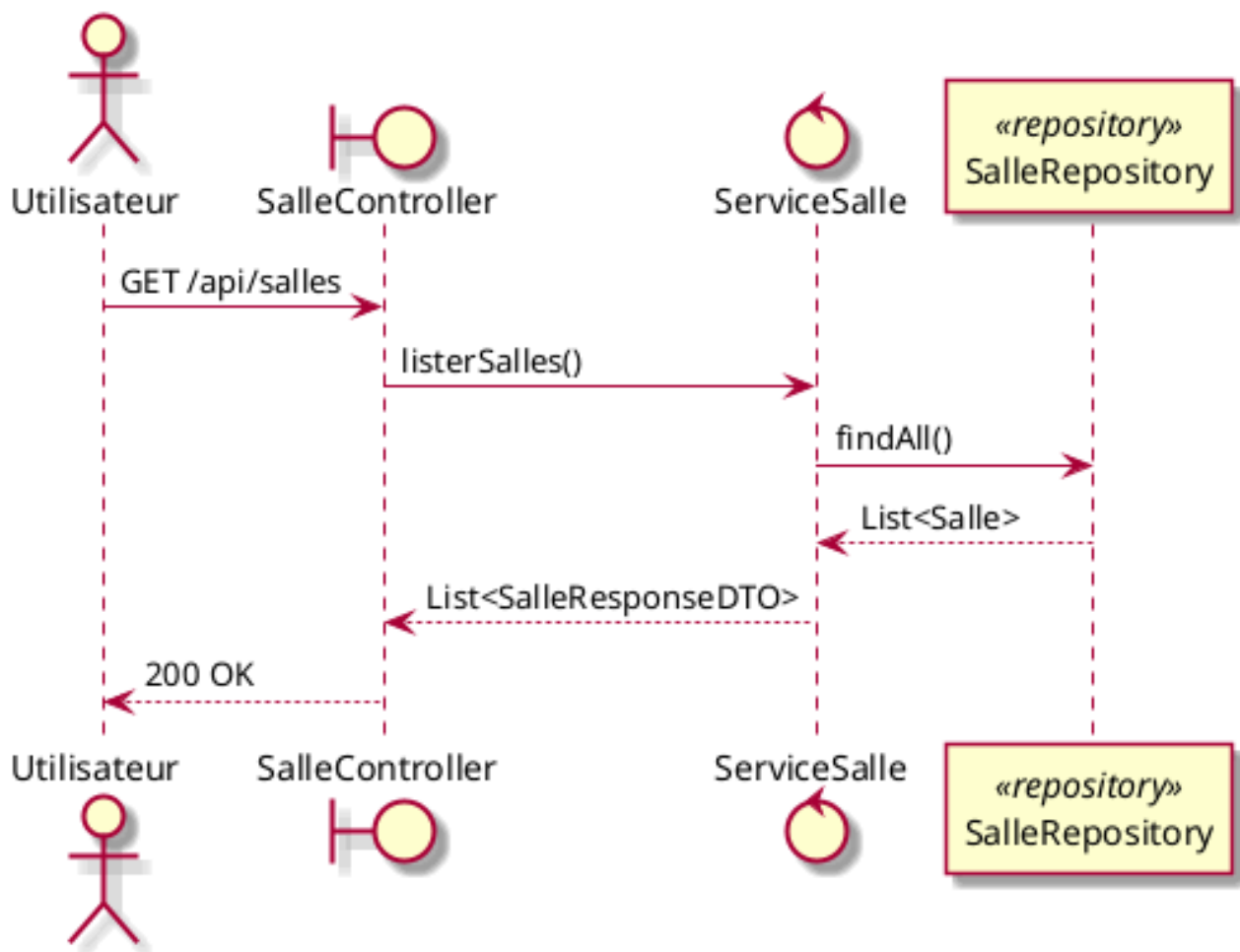


6.3.4. Consulter la liste des salles Endpoint : GET /api/salles (accès authentifié)

Classes de conception Aucune nouvelle classe. **ServiceSalle** expose une méthode de listage simple.



Séquence



6.4. Groupe 3 : Gestion des réservations

Dans l'analyse, la classe `DemandeReservation` était distincte de `Reservation`, chacune avec son propre état (`EtatDemandeReservation` et `EtatReservation`). En conception, on simplifie : une seule entité `Reservation`, dont le champ `etat` fusionne les deux enums et couvre tout le cycle de vie (`EN_ATTENTE_VALIDATION` → `CONFIRMEE` ou `REJETEE`, ou `ANNULEE`).

Les cas de ce groupe s'appuient sur `ReservationController` et `ServiceReservation`, à l'exception de la recherche de salle qui relève de `SalleController` et `ServiceSalle`.

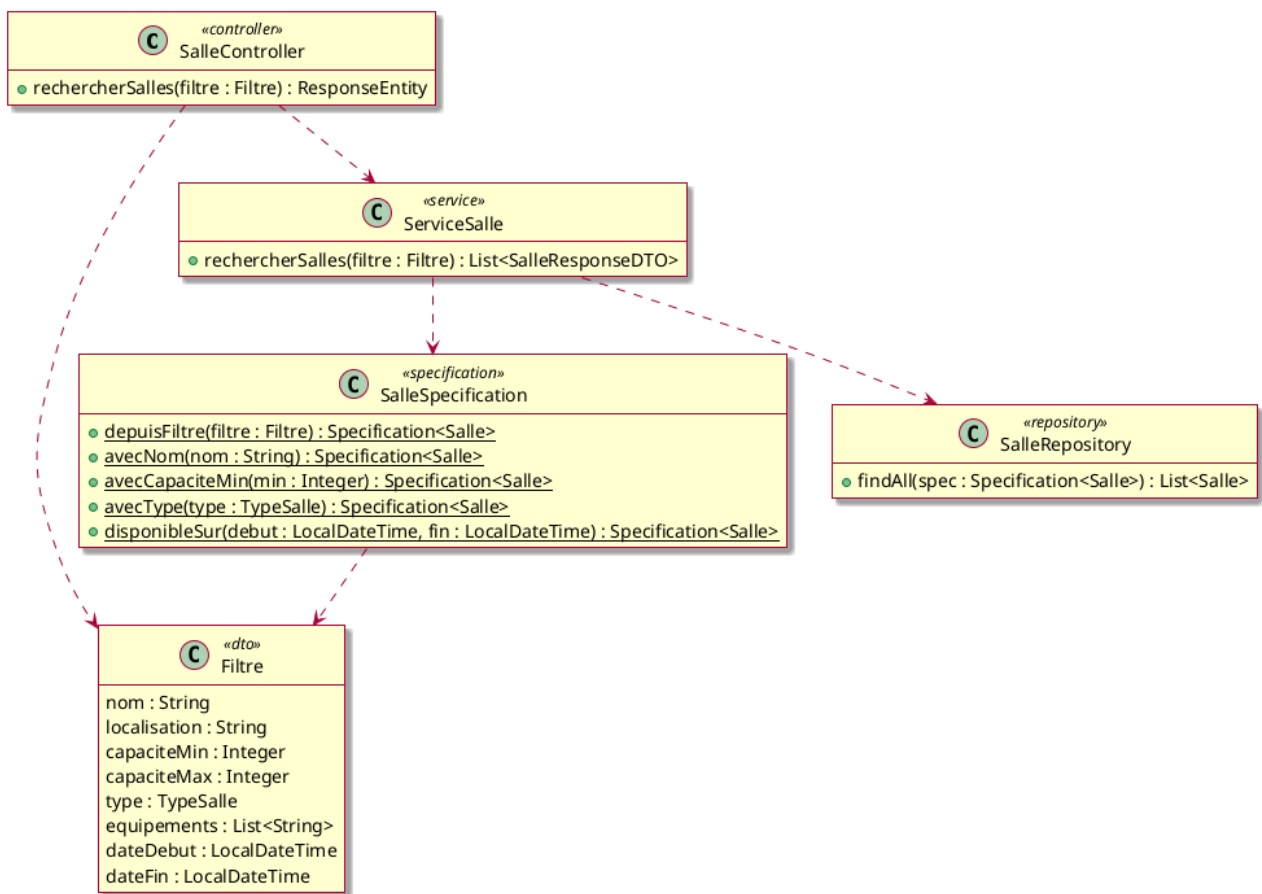
6.4.1. Rechercher une salle selon différents critères Endpoint : `POST /api/salles/recherche` (corps de requête : un objet `Filtre`)

Plutôt que de multiplier les paramètres de requête (`?nom=...&capaciteMin=...&...`), la recherche passe par un **POST** dont le corps porte un **objet unique Filtre**. Ce choix présente deux avantages :

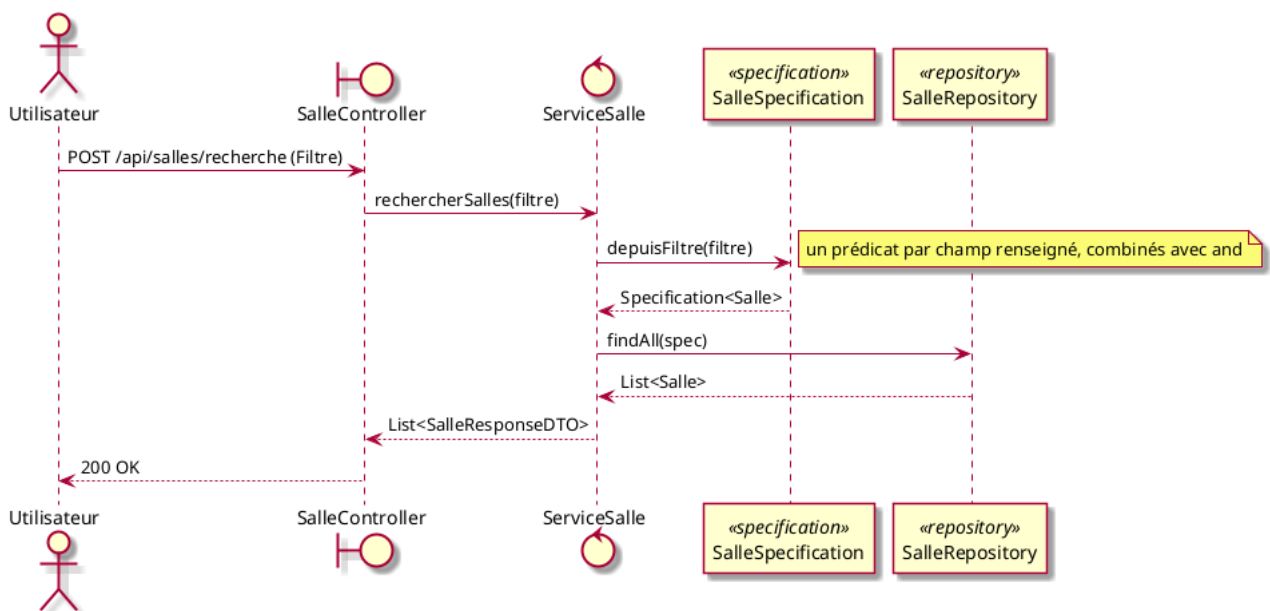
- **signature d'endpoint stable** : l'URL ne change pas quand on enrichit les critères ; seul le schéma de `Filtre` évolue ;
- **Filtre évolutif** : ajouter un critère (étage, accessibilité PMR...) revient à ajouter un champ à `Filtre` et un prédicat correspondant, sans toucher à la signature de l'endpoint.

La recherche multi-critères reste implémentée via `JpaSpecificationExecutor<Salle>` et une `SalleSpecification` composable : chaque champ renseigné de `Filtre` donne lieu à un prédicat JPA indépendant, combiné avec `and`.

Classes de conception



Séquence



6.4.2. Réserver une salle Endpoint : POST /api/reservations

Selon `salle.reservationSoumiseAValidation` : - `false` → la réservation passe directement

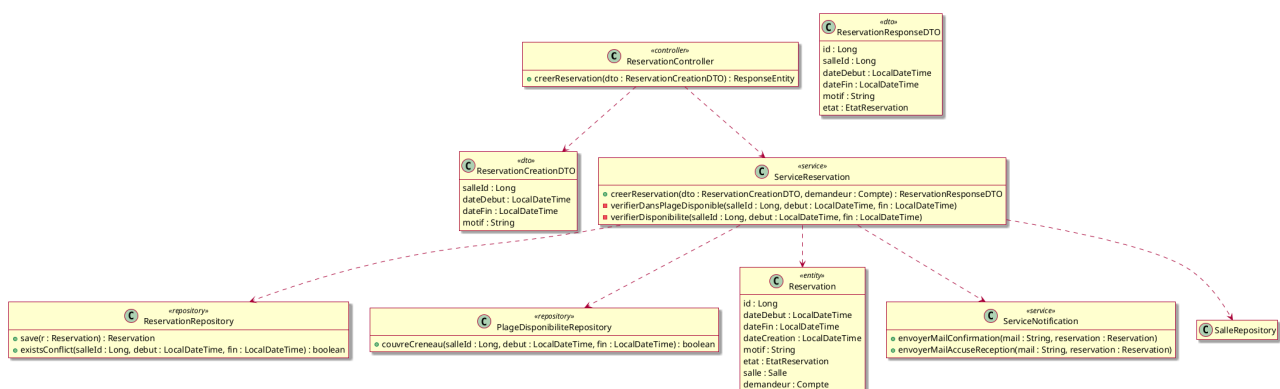
à CONFIRMEE et un mail de confirmation est envoyé ; - `true` → la réservation passe à EN_ATTENTE_VALIDATION et un mail d'accusé de réception est envoyé.

Deux vérifications précèdent la création :

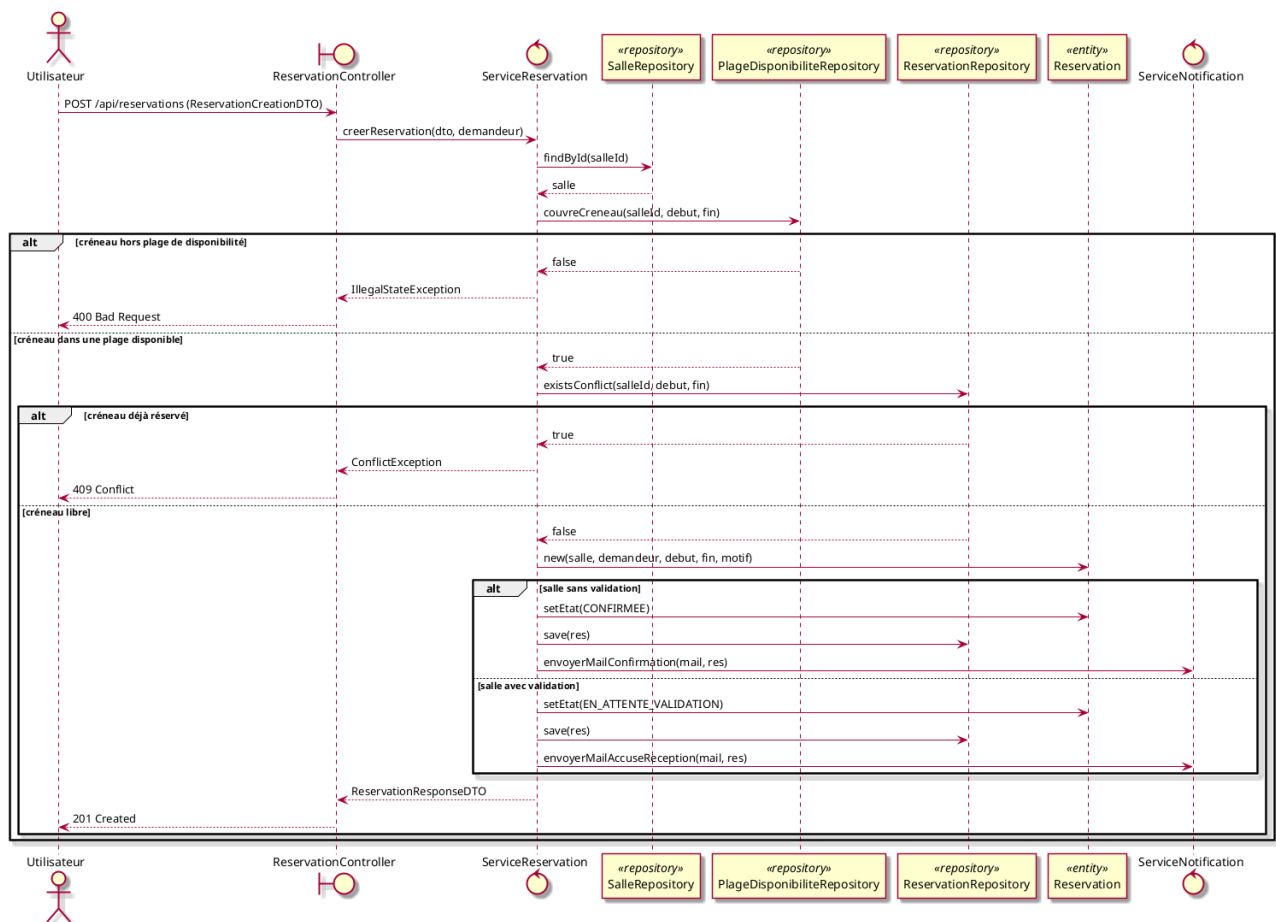
1. **Le créneau tombe dans une plage de disponibilité** de la salle : on s'assure que `[dateDebut, dateFin]` est couvert par une `PlageDisponibilite` de la salle. Sans cela, le concept de `PlageDisponibilite` serait inopérant : une salle pourrait être réservée en dehors de tout créneau déclaré ouvert.
2. **Aucun conflit avec une réservation existante** : même logique de chevauchement que pour les plages, en excluant les réservations `ANNULEE` et `REJETEE` (le filtre sur l'état fait partie de la requête `existsConflict`).

Concurrence (mécanisme simple, v1) : on ne verrouille pas le créneau pendant que le client réfléchit. Le déroulement se fait en deux temps : le client consulte la liste des salles disponibles, puis il soumet sa réservation. Si, entre les deux, un autre client a réservé le même créneau, le contrôle `existsConflict` au moment de la soumission échoue et la demande est rejetée (409 Conflict). C'est donc le premier à enregistrer qui l'emporte ; le second est informé que le créneau n'est plus libre et peut relancer une recherche.

Classes de conception



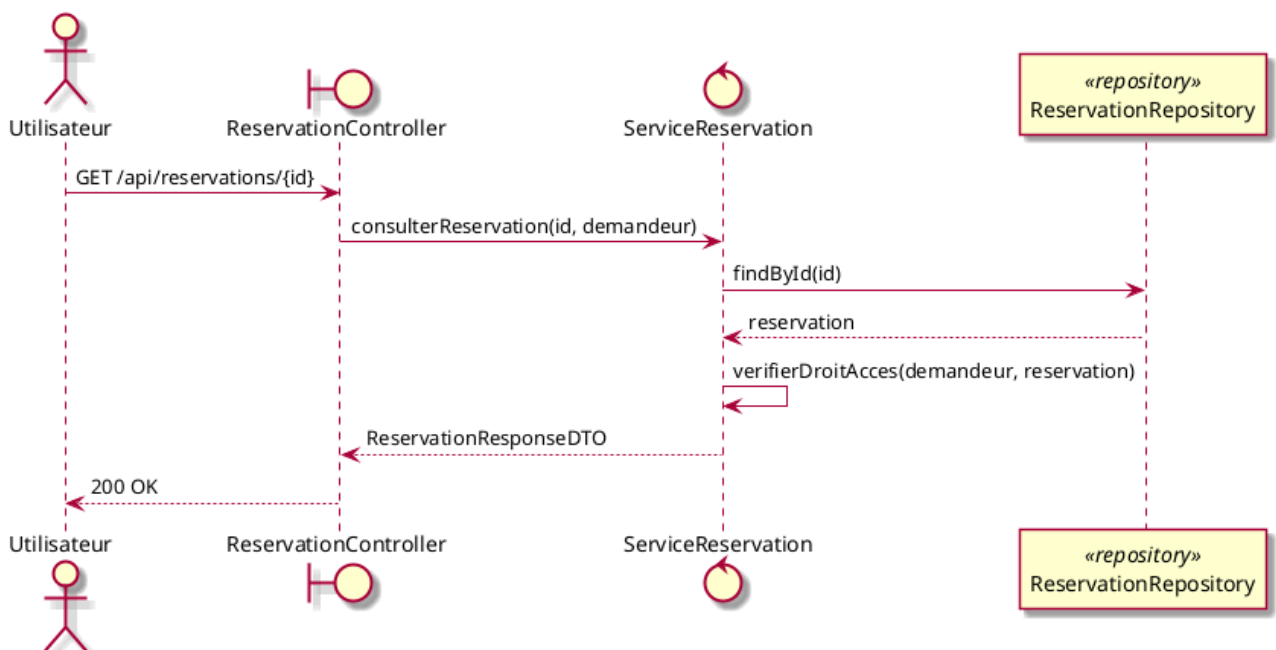
Séquence



6.4.3. Consulter une réservation Endpoint : GET /api/reservations/{id} (réservé au demandeur ou à un responsable)

Classes de conception Aucune nouvelle classe.

Séquence

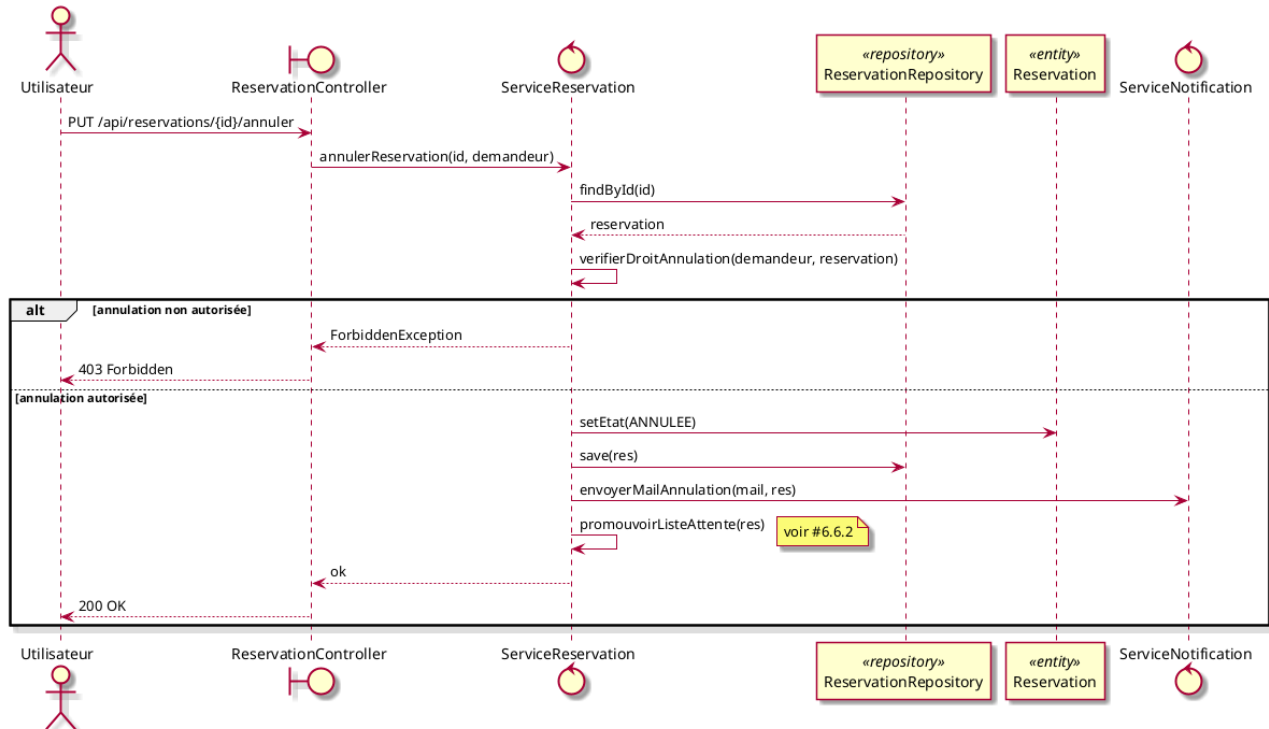


6.4.4. Annuler une réservation Endpoint : PUT /api/reservations/{id}/annuler (réservé au demandeur)

Seules les réservations en état CONFIRMEE ou EN_ATTENTE_VALIDATION peuvent être annulées.

Classes de conception Aucune nouvelle classe.

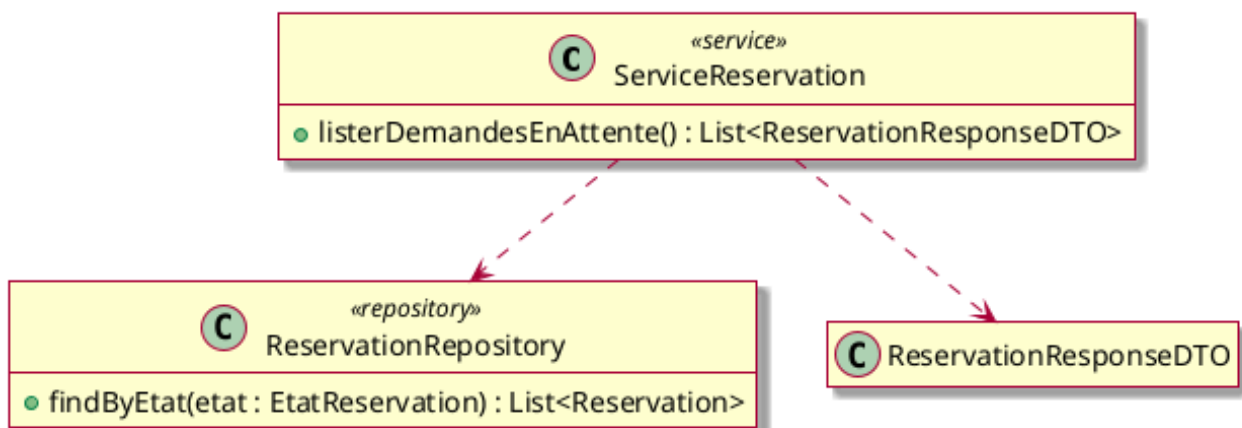
Séquence



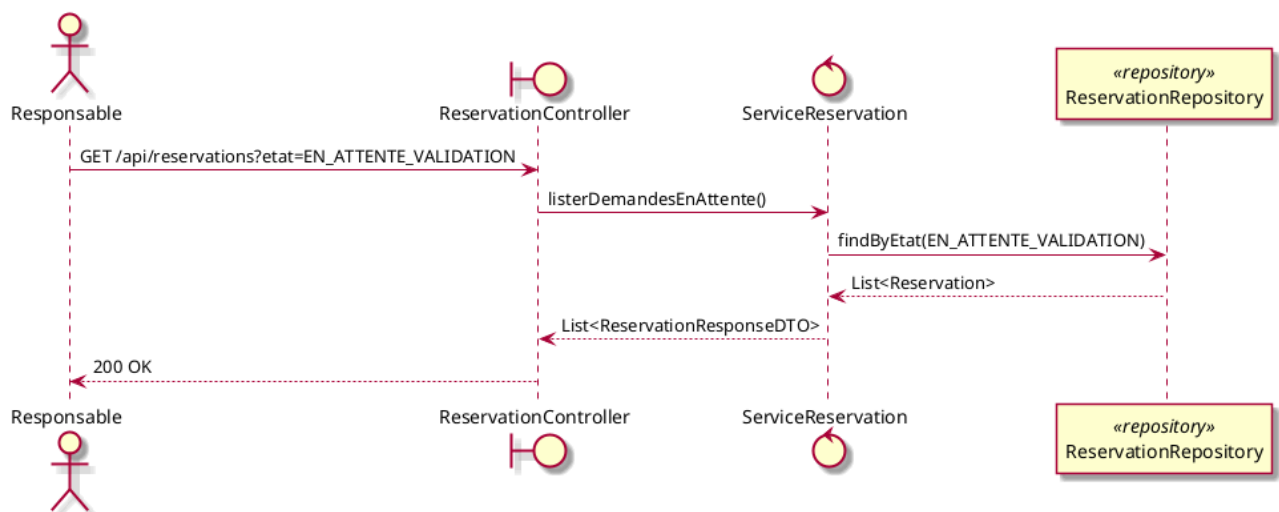
6.4.5. Consulter les demandes de réservation en attente Endpoint : GET /api/reservations?etat=EN_ATTENTE_VALIDATION (accès réservé aux responsables)

Ce cas permet à un responsable de lister les réservations qu'il doit traiter avant de les valider ou de les rejeter. Il s'appuie sur `ReservationRepository.findById`.

Classes de conception Aucune nouvelle classe. On ajoute une méthode de listage à `ServiceReservation`.



Séquence

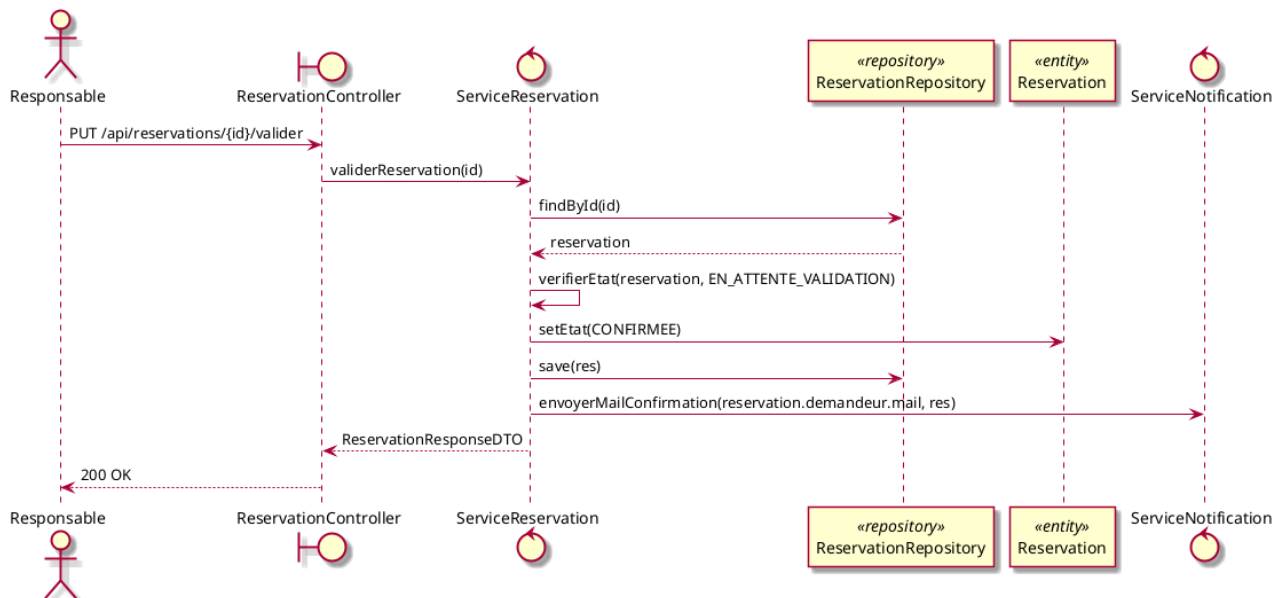


6.4.6. Valider une demande de réservation Endpoint : PUT /api/reservations/{id}/valider (accès réservé aux responsables)

Seules les réservations en état EN_ATTENTE_VALIDATION peuvent être validées.

Classes de conception Aucune nouvelle classe.

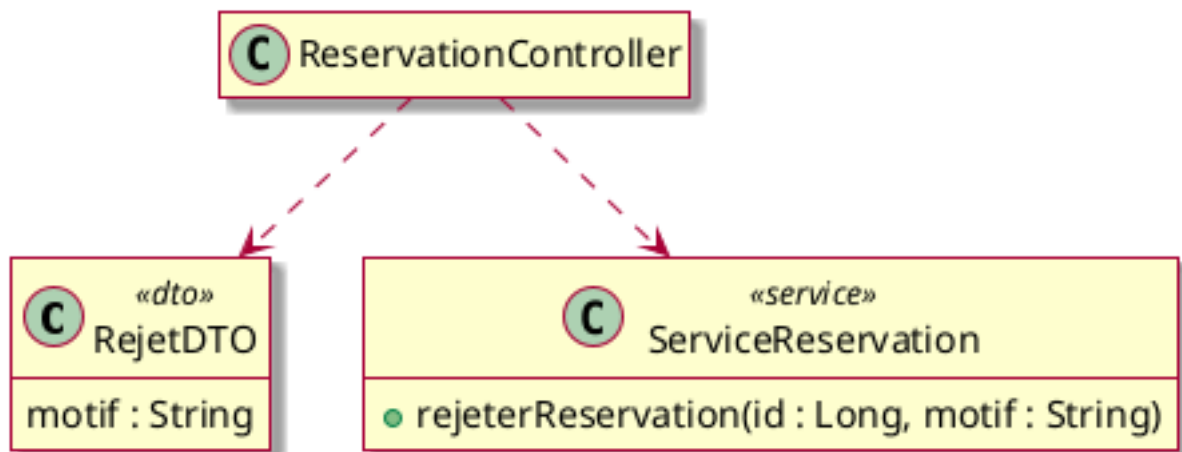
Séquence



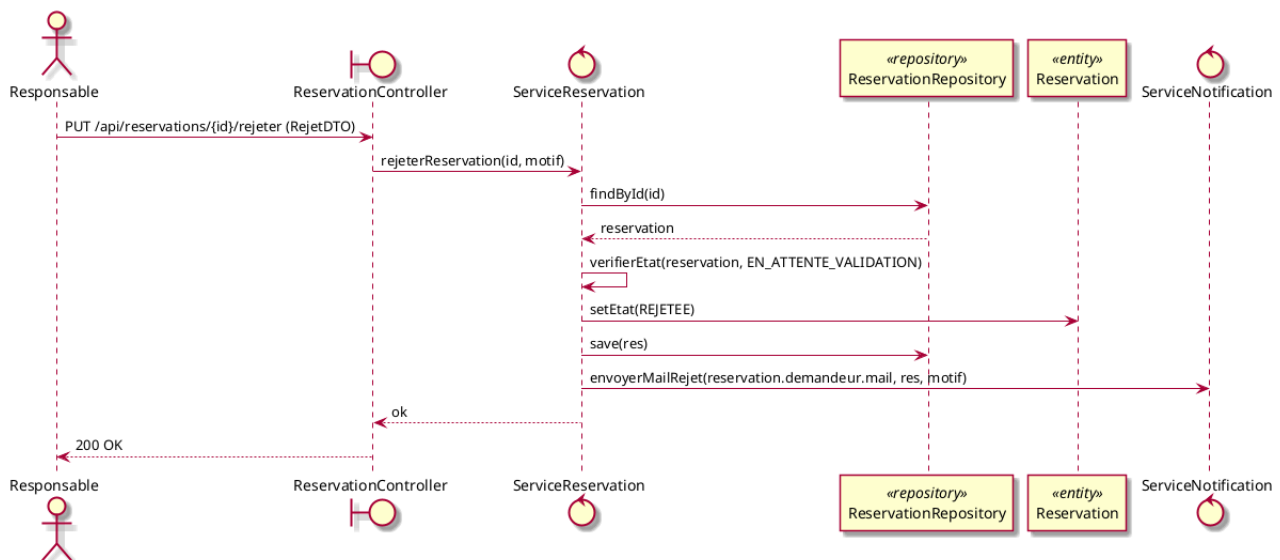
6.4.7. Rejeter une demande de réservation Endpoint : PUT /api/reservations/{id}/rejeter (accès réservé aux responsables)

Un motif de rejet est attendu dans le corps de la requête.

Classes de conception



Séquence



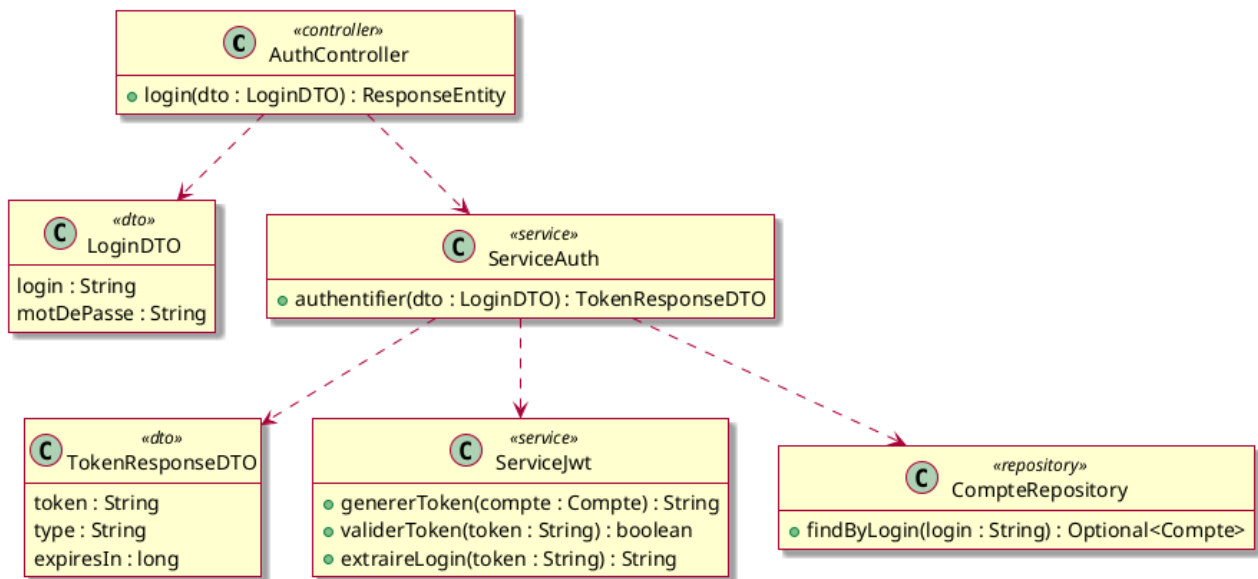
6.5. Groupe 4 : Authentification

Ce groupe couvre le cas d'utilisation "Se connecter / S'authentifier". Il s'appuie sur **AuthController**, **ServiceAuth** et **ServiceJwt**. Un **JwtFilter** (filtre Spring Security) valide le token sur chaque requête protégée ; ce composant est une infrastructure transversale et n'est pas détaillé ici.

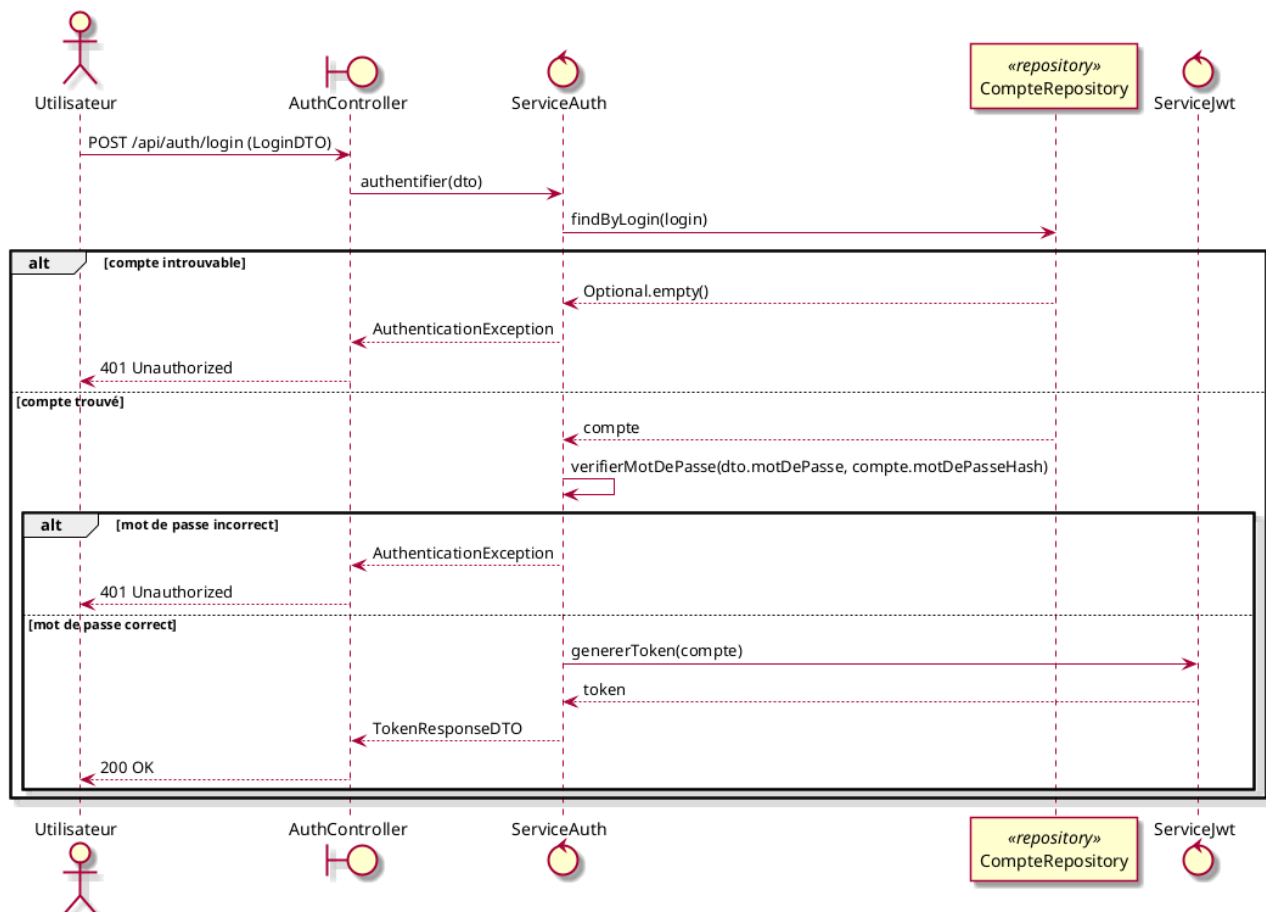
6.5.1. Se connecter / S'authentifier Endpoint : POST /api/auth/login

L'utilisateur soumet ses identifiants. **ServiceAuth** retrouve le compte via **CompteRepository**, vérifie le mot de passe (BCrypt), puis demande à **ServiceJwt** de générer un token JWT. En cas de succès, un **TokenResponseDTO** est retourné ; le client l'inclura dans l'en-tête **Authorization: Bearer <token>** de toutes les requêtes suivantes. Les erreurs d'authentification (compte introuvable ou mot de passe incorrect) retournent systématiquement un **401 Unauthorized** sans détailler la cause, pour éviter toute fuite d'information.

Classes de conception



Séquence



6.6. Groupe 5 : Gestion de la liste d'attente

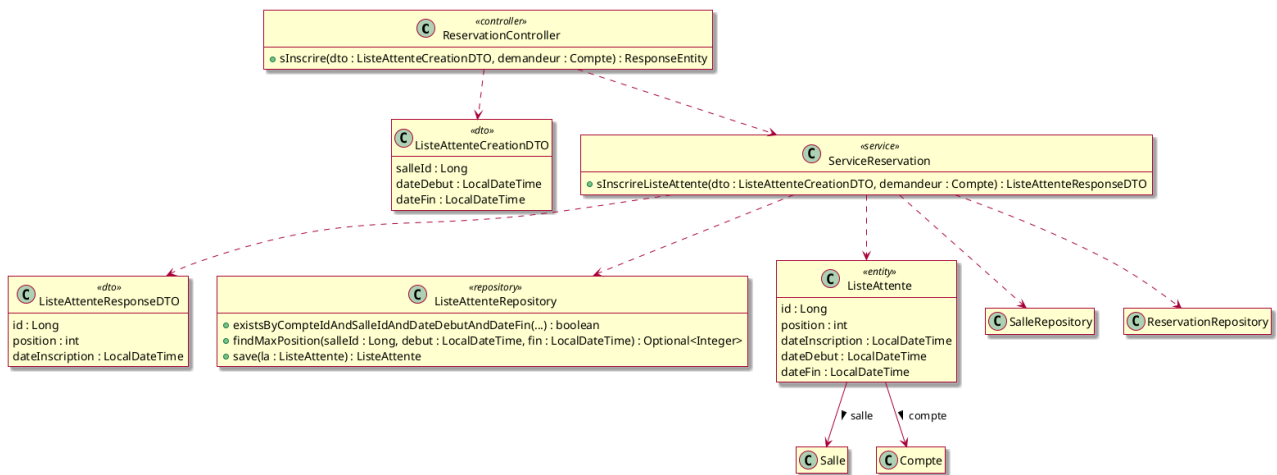
La liste d'attente s'applique uniquement aux salles à réservation directe (`reservationSoumiseAValidation` = `false`). Elle permet à un utilisateur de prendre automatiquement la place d'un autre en cas de désistement.

Note sur l'appariement des créneaux : contrairement à la détection de conflit de réservation, qui raisonne par *chevauchement* ($\text{dateDebut} < \text{fin AND dateFin} > \text{debut}$), la liste d'attente apparie les créneaux par **égalité exacte** des bornes (*salle*, *dateDebut*, *dateFin*). On ne met donc en file d'attente que des demandes portant exactement sur le même créneau qu'une réservation existante, et la promotion ne concerne que les inscrits dont le créneau coïncide avec celui libéré. C'est une simplification assumée pour la v1 : la généralisation à des créneaux chevauchants (promouvoir le premier inscrit *compatible* avec le créneau libéré) est laissée en évolution.

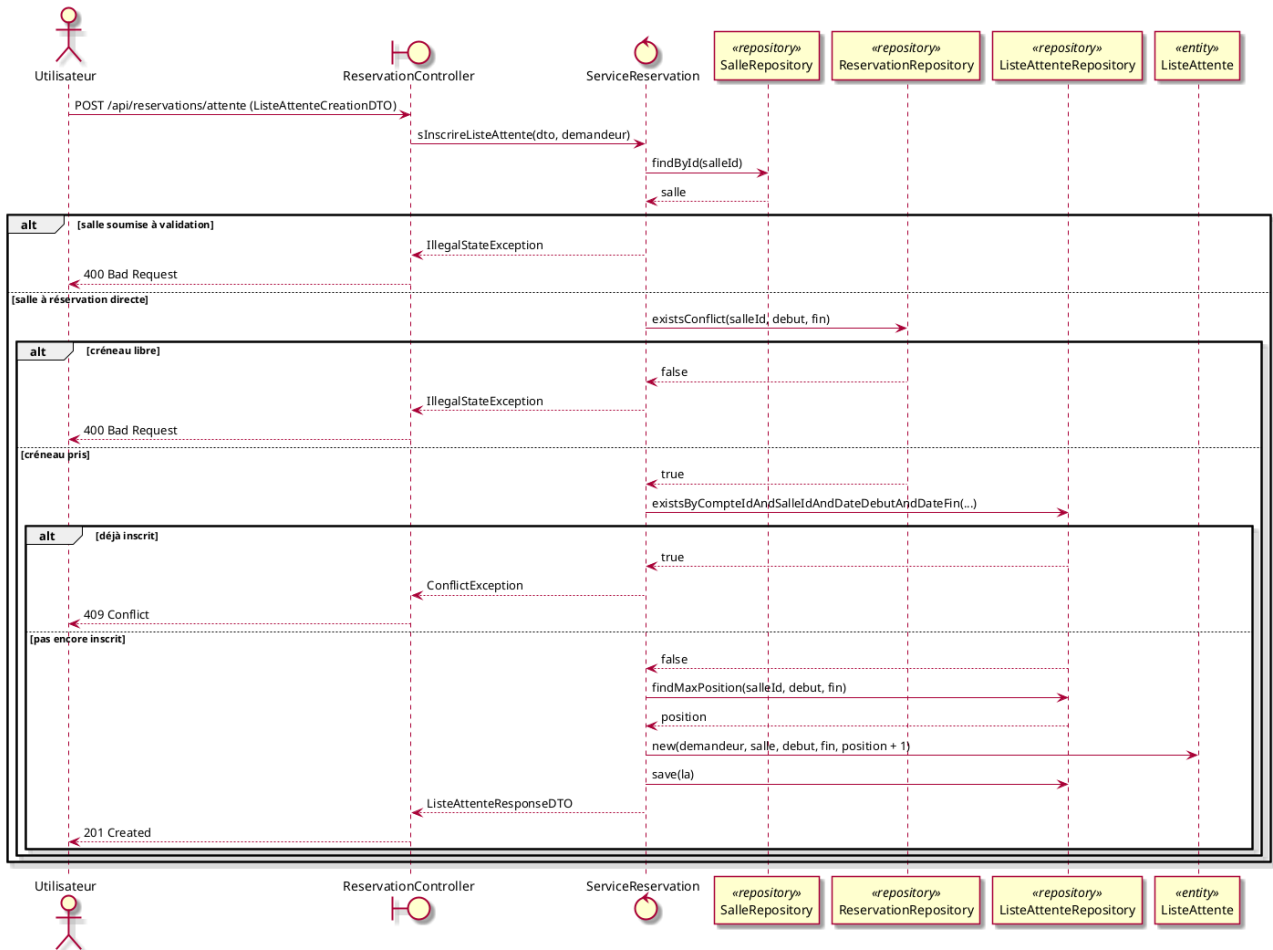
6.6.1. S'inscrire en liste d'attente Endpoint : POST /api/reservations/attente

Lorsqu'un créneau est déjà pris, l'utilisateur peut rejoindre la liste d'attente. Le système vérifie que la salle est bien à réservation directe, que le créneau est effectivement occupé, et que l'utilisateur n'y est pas déjà inscrit. La position est attribuée en fin de liste ($\text{MAX}(\text{position}) + 1$).

Classes de conception

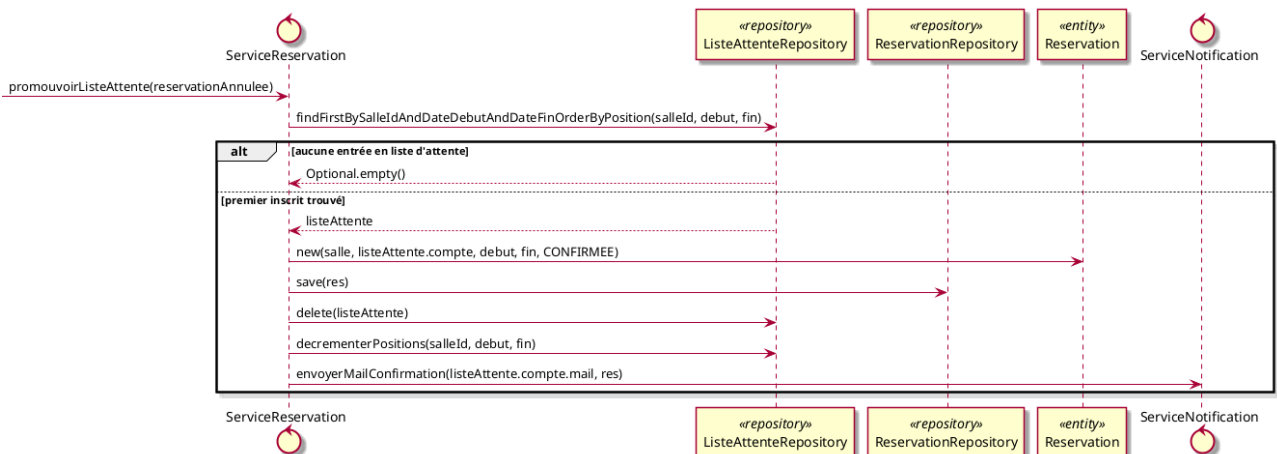


Séquence



6.6.2. Promotion automatique lors d'une annulation Ce cas n'est pas déclenché directement par un utilisateur : il est appelé par `ServiceReservation.annulerReservation` après chaque annulation. Pour les salles soumises à validation, la liste d'attente est vide par construction, l'inscription y est refusée et l'appel est sans effet. Si des inscrits existent pour le créneau libéré, le premier en liste est automatiquement promu : une **Reservation CONFIRMÉE** est créée en son nom, son entrée est retirée de la liste d'attente, les positions des inscrits suivants sont décrémentées, et un mail de confirmation lui est envoyé.

Séquence (appelée depuis `annulerReservation`)

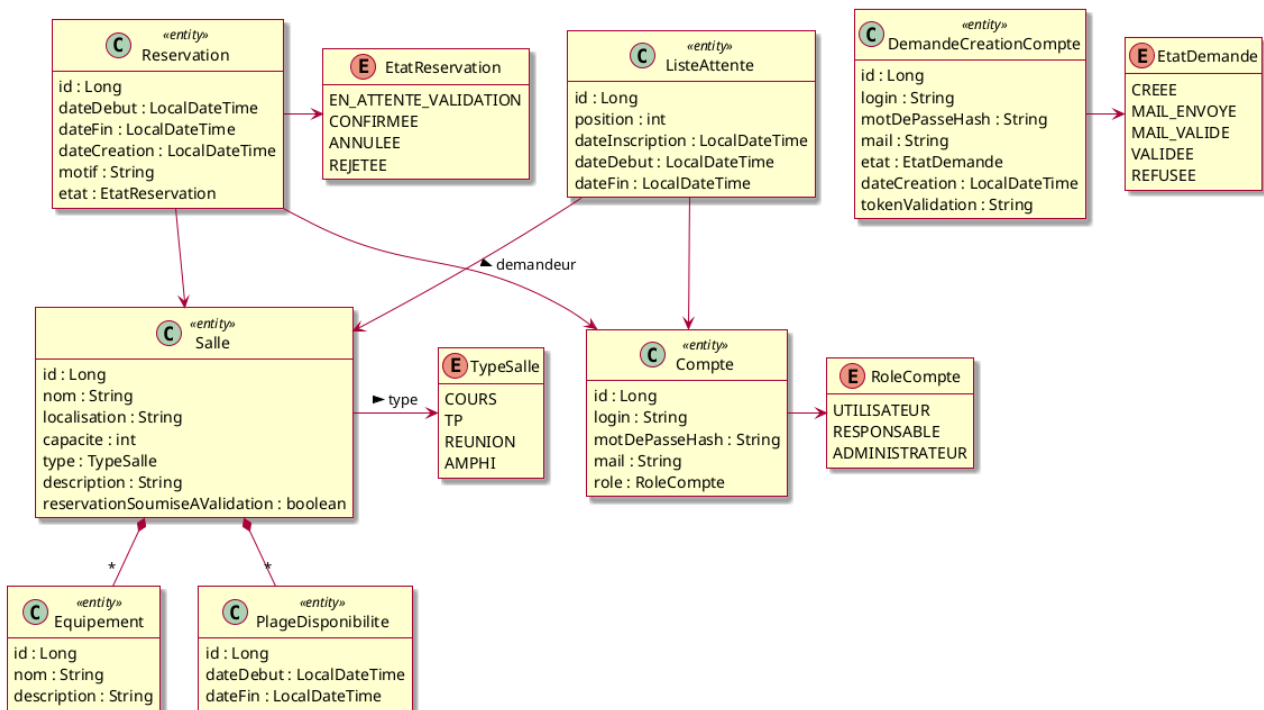


7. Regroupement des classes

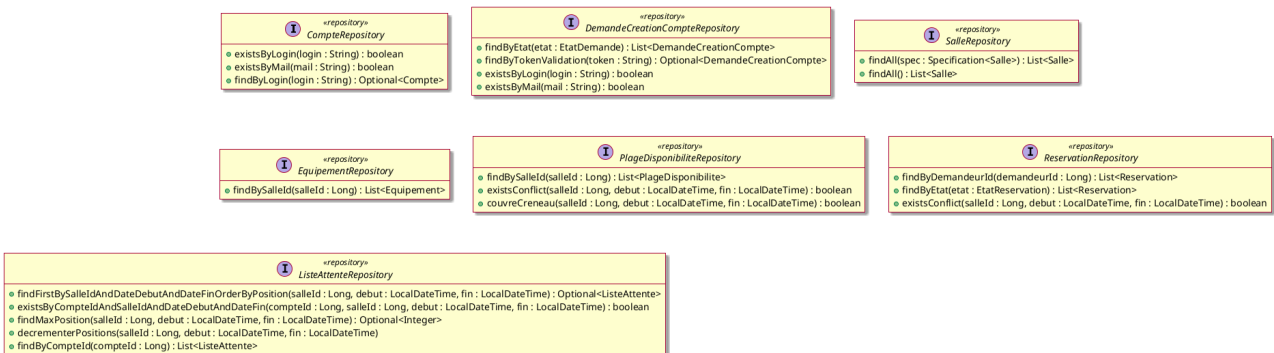
7.1. Groupe domaine

Le modèle de domaine final par rapport à l'analyse : - `DemandeReservation` est supprimée et absorbée par `Reservation.etat` : les enums `EtatDemandeReservation` et `EtatReservation` de l'analyse sont fusionnés en un seul `EtatReservation` (`EN_ATTENTE_VALIDATION`, `CONFIRMEE`, `REJETEE`, `ANNULEE`) ; - les couples `Date` + `Heure` de l'analyse sont remplacés par `LocalDateTime` (plus de type valeur `Heure` séparé), pour `PlageDisponibilite` comme pour `Reservation` ; - `DemandeCreationCompte` ajoute `tokenValidation` pour la validation par mail ; le mot de passe est stocké haché (`motDePasseHash`) ; - `Salle` conserve son type (`TypeSalle`) issu de l'analyse : il sert de critère de recherche ; l'attribut `reservationAvecValidation` est renommé `reservationSoumiseAValidation` ; - `ListeAttente` est introduite en conception pour gérer les désistements sur les salles à réservation directe.

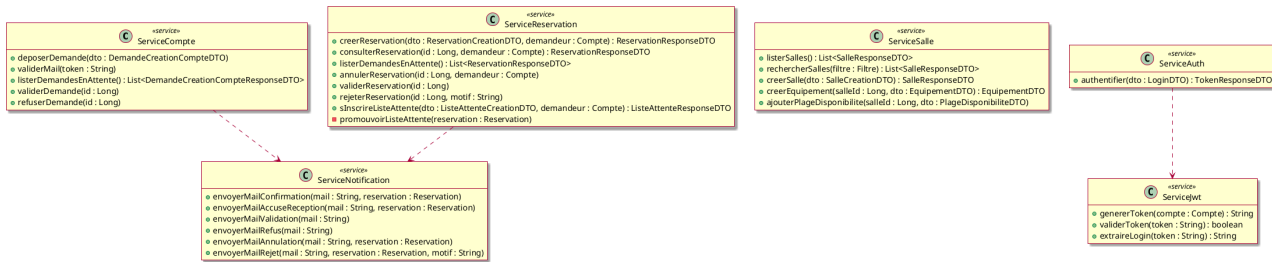
Modèle de domaine (conception)



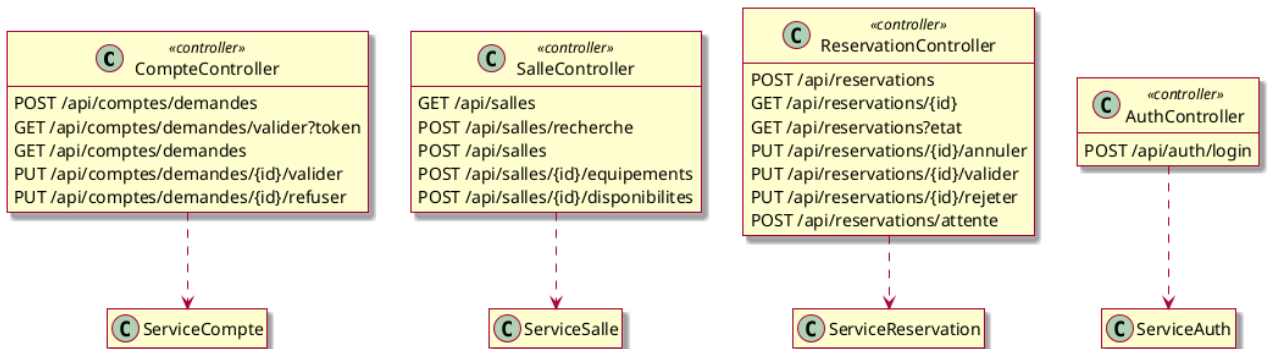
7.2. Groupe repositories



7.3. Groupe services



7.4. Groupe contrôleurs REST



8. Choix, questions ouvertes et remarques

- **Gestion des conflits de réservation** : la vérification qu'une salle est libre sur un créneau donné est une requête potentiellement complexe. Il faudra définir précisément la requête JPA ou SQL correspondante.
- **Concurrence sur la création de réservation** : pour la v1, on retient un mécanisme simple et optimiste : la disponibilité est consultée puis la réservation soumise séparément, et toute demande arrivant après qu'un autre client a pris le créneau est rejetée (409 Conflict). Le premier à enregistrer l'emporte, sans verrou ni session réservant le créneau. Une évolution pourra être envisagée si une vraie atomicité devient nécessaire sous forte charge.
- **Liste d'attente sur créneaux chevauchants** : la v1 apparie les inscriptions par créneau exact. Généraliser à la promotion du premier inscrit *compatible* avec un créneau libéré (chevauchement) est une évolution à étudier.
- **Double mode de réservation** : le fait qu'une salle puisse être soit à réservation directe, soit à validation, implique deux branches dans le cycle de vie de **Reservation**. Le *Design Pattern State* pourrait être envisagé pour gérer proprement ces deux comportements.
- **Annulation automatique** : quand une salle devient indisponible (travaux, événement prioritaire), les réservations existantes doivent être annulées automatiquement et les utilisateurs notifiés.
- **JWT et sécurité** : il faudra préciser la durée de vie des tokens, le mécanisme de refresh, et les règles CORS pour autoriser les appels depuis le frontend Vue.js.